arXiv:2510.23972v2 [cs.LG] 10 Dec 2025

An efficient probabilistic hardware architecture for diffusion-like models

Andraž Jelinčič,[1,*] Owen Lockwood,[1] Akhil Garlapati,[1] Peter Schillinger,[1]
Isaac L. Chuang,[2] Guillaume Verdon,[1] and Trevor McCourt[*,1,†]

[1] *Extropic Corp.*
[2] *Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology*

(Dated: December 12, 2025)

The proliferation of probabilistic AI has prompted proposals for specialized stochastic computers. Despite promising efficiency gains, these proposals have failed to gain traction because they rely on fundamentally limited modeling techniques and exotic, unscalable hardware. In this work, we address these shortcomings by proposing an all-transistor probabilistic computer that implements powerful denoising models at the hardware level. A system-level analysis indicates that devices based on our architecture could achieve performance parity with GPUs on a simple image benchmark using approximately 10,000 times less energy.

The unprecedented recent investment in large-scale AI systems will soon put a strain on the world's energy infrastructure. Every year, U.S. firms spend an amount larger than the inflation-adjusted cost of the Apollo program on AI-focused data centers [1, 2]. By 2030, these data centers could consume around 10% of all of the energy produced in the U.S. [3]. At the same time, model sizes, dataset sizes, and deployment footprints continue to grow rapidly.

Existing AI systems based on autoregressive large language models (LLMs) are valuable tools in white-collar fields [4–9], and are being adopted by consumers faster than the internet [10]. However, LLMs were architected specifically for GPUs [11], hardware originally intended for graphics, whose suitability for machine learning was discovered accidentally decades later [12, 13]. Had a different style of hardware been popular in the last few decades, AI algorithms would likely have evolved in a different direction, and possibly a more energy-efficient one. This interplay between algorithm research and hardware availability is known as the "hardware lottery" [14], and it entrenches hardware–algorithm pairings that may be far from optimal.

Therefore, prudent planning calls for systematic exploration of other types of AI systems in search of energy-efficient alternatives. Active efforts include mixed-signal compute-in-memory accelerators [15], photonic neural networks [16], and neuromorphic processors that emulate biological spiking [17, 18]. These approaches explicitly confront the fact that computation is a physical process and that respecting physical constraints can lead to qualitatively different designs.

Probabilistic computing is an attractive approach because it can connect directly to AI at the system level via Energy-Based Models (EBMs). EBMs are a well-established model class in contemporary deep learning and have been competitive with the state of the art in tasks like image generation and robotic path planning [19, 20]. Using probabilistic hardware to accelerate EBMs falls under the broad umbrella of *thermodynamic computing* [21], in which computation is expressed in terms of stochastic dynamics on energy landscapes.

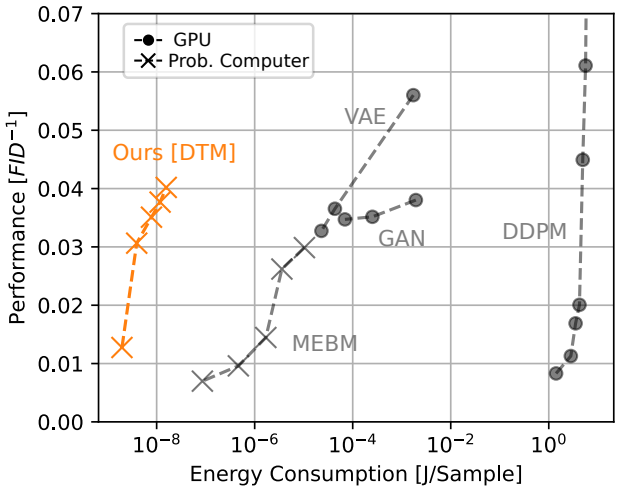

FIG. 1. **Leveraging CMOS probabilistic hardware in ultra-efficient AI systems.** The central result of this article: an all-transistor probabilistic computer running a denoising thermodynamic model (DTM) could match GPU performance on a simple modeling benchmark while using about 10,000× less energy. All models are trained on binarized Fashion-MNIST [22] and evaluated with Fréchet Inception Distance (FID) [23]. DTM variants are of increasing depth, chaining 2–8 sequential Energy-Based Models (EBMs). GPU baselines cover single-step VAE [24] and GAN [25], plus DDPM [26] at varying numbers of steps. We also compare DTM to a monolithic EBM across multiple mixing-time limits. The horizontal axis shows the energy needed for generating a single new image using the trained model (inference).

Hardware implementations of EBMs work with special model families that adhere to physical constraints such as locality, sparsity, and limited connection density. Thanks to these constraints, probabilistic computers can utilize specialized stochastic circuitry to efficiently and quickly produce samples from a Boltzmann distribution [27]. De-

* These authors contributed equally to this work.
† Corresponding author: trevor@extropic.ai

pending on the precise kind of hardware being used, this sampling may occur as part of the natural dynamics of the device [28–32] or may be orchestrated using an algorithm like Gibbs sampling [33–36].

Past attempts at EBM accelerators have suffered from issues at both the architectural and hardware levels. All previous proposals used EBMs as monolithic models of data distributions, which is known to be challenging to scale [37]. Additionally, existing devices have relied on exotic components such as magnetic tunnel junctions as sources of intense thermal noise for random-number generation (RNG) [35, 38, 39]. These exotic components have not yet been tightly integrated with transistors in commercial CMOS processes and do not currently constitute a scalable solution [40–42].

In this work, we address these issues and propose a commercially viable probabilistic computing system that is designed for thermodynamic hardware from the outset. At the modeling level, we introduce Denoising Thermodynamic Models (DTMs), which repurpose hardware EBMs as denoising steps rather than monolithic models of the data distribution, thereby mitigating the mixing–expressivity tradeoff that plagues monolithic EBMs. At the architectural level, we propose the Denoising Thermodynamic Computer Architecture (DTCA), which tightly integrates DTMs into probabilistic hardware built from sparse, locally connected Boltzmann machines and an all-transistor RNG. Finally, we perform a system-level energy analysis, using measurements from an experimental thermodynamic RNG chip together with physical models and simulations, and show that a device based on this architecture could match the performance of GPU-based generative models on binarized Fashion-MNIST (as measured by Fréchet Inception Distance) while using approximately 10,000 times less energy per generated sample.

The remainder of this article will substantiate the results presented in Fig. 1, which are based on a combination of measurements from real circuits, physical models, and simulations. To begin, we introduce a fundamental compromise inherent in using EBMs as standalone models of data, which we refer to as the *mixing–expressivity tradeoff*. We then discuss how this compromise can be avoided by wielding EBMs as part of a denoising process rather than monolithically. Next, we outline how to build a hardware system using DTMs to implement this denoising process at a very low level. Then, we study simulations of this hardware system, further justifying the results shown in Fig. 1 and highlighting some of the practical merits of DTMs compared to existing approaches. Finally, we conclude by discussing how the capabilities of probabilistic accelerators for machine learning may be scaled by merging them with traditional neural networks (NNs). We demonstrate that such a hybrid model composed of a DTM and a neural network can achieve performance parity with a purely NN-based model on the CIFAR-10 benchmark while using a ten times smaller neural network component (both in terms of the number of parameters and FLOPs) and thus consuming roughly an order of magnitude less energy in total.

I. THE CHALLENGE WITH EBMS

The fundamental problem of machine learning is inferring the probability distribution that underlies some data [43, 44]. An early approach [45] to this was to use a monolithic EBM (MEBM) to fit a data distribution directly by shaping a parameterized energy function $\mathcal{E}$:

$$P(x;\theta) \propto e^{-\mathcal{E}(x,\theta)}, \tag{1}$$

where x is a random variable representing the data and θ represents the parameters of the EBM.

Fitting an MEBM corresponds to assigning low energies to values of x where data are abundant and high energies to values of x that are far from data. Real-world data are often clustered into distinct modes [46, 47], meaning that an MEBM that fits data well will have a complex, rugged energy landscape with many deep valleys surrounded by tall mountains. This complexity is illustrated by the cartoon in Fig. 2 (a).

Unlike the systems we propose, existing probabilistic computers based on MEBMs struggle with the multimodality of real-world data, which hinders their efficiency. Namely, the amount of energy the computer must expend to draw a sample from the MEBM's distribution can be tremendous if its energy landscape is very rough.

Specifically, sampling algorithms that operate in high dimensions (such as Gibbs sampling [48]) are locally informed iterative procedures, meaning that they sample a landscape by randomly making small movements in the space based on low-dimensional information. When using such a procedure to sample from Eq. (1), the probability that the iteration will move up in energy to some state $X[k+1]$ is exponentially small in the energy increase compared to the current state $X[k]$, i.e.,

$$\mathbb{P}(X[k+1] = x' | X[k] = x) \propto e^{-\left(\mathcal{E}(x') - \mathcal{E}(x)\right)}. \tag{2}$$

For large differences in energy, like those encountered when trying to move between two valleys separated by a significant barrier, this probability can be very close to zero. In standard Markov-chain analyses, such barriers lead to exponentially large expected transition times between modes, which is reflected in a rapidly growing mixing time.

The mixing–expressivity tradeoff (MET) summarizes this issue with existing probabilistic computer architectures, reflecting the fact that modeling performance and sampling hardness are coupled for MEBMs. Specifically, as the expressivity (modeling performance) of an MEBM increases, its *mixing time* (the amount of computational effort needed to draw independent samples from the MEBM's distribution) becomes progressively longer, resulting in expensive inference and unstable

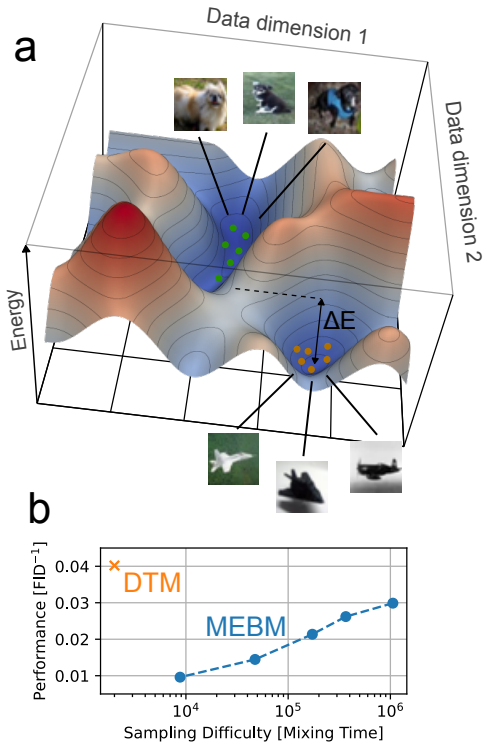

FIG. 2. **The mixing–expressivity tradeoff. (a)** A cartoon illustrating the mixing–expressivity tradeoff in EBMs. It shows a projection of an energy landscape fit to a simple dataset. The "airplane" mode is well separated from the "dog" mode, with very little data in between. Progressively better fits of the EBM to the data tend to feature larger energy barriers ΔE between the modes, making the EBM increasingly difficult to sample from. **(b)** An example of the effect of the mixing–expressivity tradeoff on model performance as measured using the Fashion-MNIST dataset. The blue curve in the plot shows the results of experiments on MEBMs with limited allowed mixing time. Performance and mixing time are strongly correlated. Mixing times were computed by fitting an exponential function to the large-lag behavior of the autocorrelation function; see Appendix L. In contrast, a DTM (orange cross) has higher performance despite substantially lower sampling requirements.

training [49, 50]; see Appendix G for a definition of mixing time.

The empirical effect of the MET on the efficiency of MEBM-based probabilistic computing systems is illustrated in Fig. 2 (b). Mixing time increases very rapidly with performance, inflating the amount of energy required to sample from the model. The effect of this increased mixing time is reflected in Fig. 1: despite the MEBM-based solution using the same EBMs and underlying hardware as the DTM-based solution, its energy consumption is several orders of magnitude larger due to the glacially slow mixing.

II. DENOISING THERMODYNAMIC MODELS

The MET makes it clear that MEBMs have a flaw that makes them challenging and energetically costly to scale. However, this flaw is avoidable, and many types of probabilistic machine learning models have been developed to solve the distribution modeling problem while circumventing the MET.

Denoising diffusion models were explicitly designed to sidestep the MET by gradually building complexity through a series of simple, easy-to-sample probabilistic transformations [26]. By doing so, they allowed for much more complex distributions to be expressed given a fixed compute budget and substantially expanded the capabilities of generative models [51–53].

DTMs merge EBMs with diffusion models, offering an alternative path for probabilistic computing that mitigates the MET. DTMs are a slight generalization of recent work from deep learning practitioners that has pushed the frontier of EBM performance [54–57].

Instead of trying to use a single EBM to model the data, DTMs chain many EBMs to gradually build up to the complexity of the data distribution. This gradual buildup of complexity allows the landscape of each EBM in the chain to remain relatively simple (and easy to sample) without limiting the complexity of the distribution modeled by the chain as a whole; see Fig. 2 (b).

Denoising models attempt to reverse a process that gradually transforms the data distribution $Q(x^0)$ into simple noise. This forward process is given by the Markov chain

$$Q(x^0, \ldots, x^T) = Q(x^0) \prod_{t=1}^{T} Q(x^t | x^{t-1}). \tag{3}$$

The forward process is typically chosen such that it has a unique stationary distribution $Q(x^T)$, which takes a simple form (e.g., Gaussian or uniform).

Reversal of the forward process is achieved by learning a set of distributions $P_\theta(x^{t-1}|x^t)$ that approximate the reversal of each conditional in Eq. (3). In doing so, we learn a map from simple noise to the data distribution, which can then be used to generate new data.

In traditional diffusion models, the forward process is made to be sufficiently fine-grained (using a large number of steps T) such that the conditional distribution of each step in the reverse process takes some simple form (such as Gaussian or categorical). This simple distribution is parameterized by a neural network, which is then trained to minimize the Kullback–Leibler (KL) divergence between the joint distributions Q and P_θ,

$$\mathcal{L}_{DN}(\theta) = D\left(Q(x^0, \ldots, x^T) \middle\| P_\theta(x^0, \ldots, x^T)\right), \tag{4}$$

where the joint distribution of the model is the product

of the learned conditionals:

$$P_\theta(x^0, \ldots, x^T) = Q(x^T) \prod_{t=1}^{T} P_\theta(x^{t-1}|x^t). \tag{5}$$

See Appendix B.2 for more details.

EBM-based denoising models approach the problem from a different angle [54]. In many cases, it is straightforward to re-cast the forward process in an exponential form,

$$Q(x^t|x^{t-1}) \propto e^{-\mathcal{E}^f_{t-1}(x^{t-1}, x^t)}, \tag{6}$$

where $\mathcal{E}^f_{t-1}$ is the energy function associated with the forward process step that adds noise to x^{t-1}. We then use an EBM with a particular energy function to model the conditional, i.e.,

$$P_\theta(x^{t-1}|x^t) \propto e^{-\left(\mathcal{E}^f_{t-1}(x^{t-1}, x^t) + \mathcal{E}^\theta_{t-1}(x^{t-1}, \theta)\right)}. \tag{7}$$

In this parameterization, the dependence on x^t enters entirely through the forward energy $\mathcal{E}^f_{t-1}(x^{t-1}, x^t)$, which constrains x^{t-1} to stay close to the noisy input x^t, while $\mathcal{E}^\theta_{t-1}$ shapes the local structure of the distribution over x^{t-1}.

Equation (7) allows for a compromise between the number of steps in the approximation to the reverse process and the difficulty of sampling at each step. As the number of steps in the forward process is increased, the effect of each noising step becomes smaller, meaning that $\mathcal{E}^f_{t-1}$ more tightly binds x^t to x^{t-1}. This binding can simplify the distribution given in Eq. (7) by imposing an energy penalty that prevents it from being strongly multimodal; see Appendix B.4 for further discussion.

As illustrated in Fig. 3 (a), models of the form given in Eq. (7) reshape simple noise into an approximation of the data distribution. Increasing T while holding the EBM architecture constant simultaneously increases the expressive power of the chain and makes each step easier to sample from, entirely bypassing the MET.

To maximally leverage probabilistic hardware for EBM sampling, DTMs generalize Eq. (7) by introducing latent variables $\{z^t\}$:

$$P_\theta(x^{t-1}|x^t) \propto \sum_{z^{t-1}} e^{-\left(\mathcal{E}^f_{t-1}(x^{t-1}, x^t) + \mathcal{E}^\theta_{t-1}(x^{t-1}, z^{t-1}, \theta)\right)}. \tag{8}$$

Introducing latent variables allows the size and complexity of the probabilistic model to be increased independently of the data dimension. In this generalized form, $\mathcal{E}^f_{t-1}$ still enforces proximity between x^{t-1} and x^t, while $\mathcal{E}^\theta_{t-1}$ uses the latent variables z^{t-1} to enrich the local conditional structure without directly depending on x^t.

A convenient property of DTMs is that if the approximation to the reverse-process conditional is exact ($P_\theta(x^{t-1}|x^t) \to Q(x^{t-1}|x^t)$), one also learns the marginal distribution at $t-1$,

$$Q(x^{t-1}) \propto \sum_{z^{t-1}} e^{-\mathcal{E}^\theta_{t-1}(x^{t-1}, z^{t-1}, \theta)}. \tag{9}$$

See Appendix B.6 and Ref. [54] for further details and discussion of this property. Note that it relies on the normalizing constant associated with the distribution in Eq. (6) being independent of x^{t-1}.

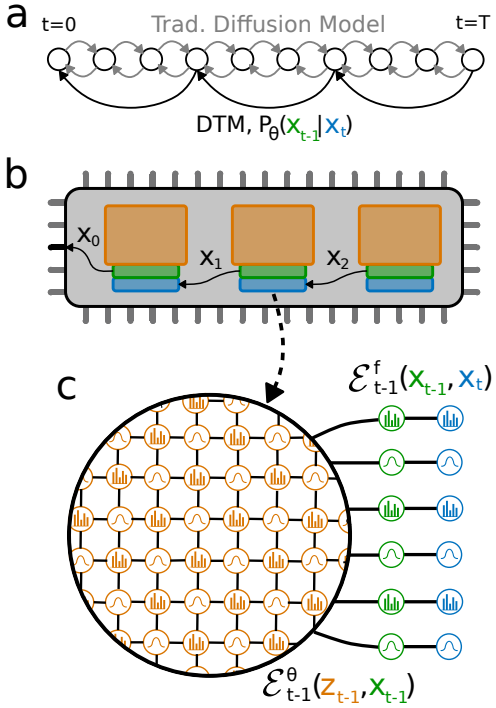

FIG. 3. **The denoising thermodynamic computer architecture.** **(a)** Traditional diffusion models have simple conditionals and must take small steps when approximating the reverse process. Since EBMs can express more complex distributions, DTMs can take potentially much larger steps. **(b)** A sketch of how a chip based on the DTCA chains hardware EBMs to approximate the reverse process. Each EBM is implemented by distinct circuitry, parts of which are dedicated to receiving the inputs and conditionally sampling the outputs and latents. **(c)** An abstract diagram of a hardware EBM. The state variables x^t and x^{t-1} map onto distinct physical degrees of freedom represented by the blue and green nodes, respectively. The coupling between these two sets of nodes implements the forward process energy function $\mathcal{E}^f_t\left(x^{t-1}, x^t\right)$. The set of orange nodes represents a set of latent variables z^{t-1}. The couplings between these nodes and to the x^{t-1} nodes implements $\mathcal{E}^\theta_{t-1}\left(z^{t-1}, x^{t-1}\right)$.

III. DENOISING THERMODYNAMIC COMPUTERS

The Denoising Thermodynamic Computer Architecture (DTCA) tightly integrates DTMs into probabilistic hardware, allowing for the highly efficient implementa-

tion of EBM-aided diffusion models.

Practical implementations of the DTCA utilize natural-to-implement EBMs that exhibit sparse and local connectivity, as is typical in the literature [33]. This constraint allows sampling of the EBM to be performed by massively parallel arrays of primitive circuitry that implement Gibbs sampling. Refer to Appendices C and D for further theoretical discussion of the hardware architecture.

A key feature of the DTCA is that $\mathcal{E}^f_{t-1}$ can be implemented efficiently using our constrained EBMs. Specifically, for both continuous and discrete diffusion, $\mathcal{E}^f_{t-1}$ can be implemented using a single pairwise interaction between corresponding variables in x^t and x^{t-1}; see Appendices B.1 and D.1 for details. This structure can be reflected in how the chip is laid out to implement these interactions without violating locality constraints.

Critically, Eq. (8) places no constraints on the form of $\mathcal{E}^\theta_{t-1}$. Therefore, we are free to use EBMs that our hardware implements especially efficiently. At the lowest level, this corresponds to high-dimensional, regularly structured latent-variable EBMs. If more powerful models are desired, these hardware latent-variable EBMs can be arbitrarily scaled by combining them into software-defined graphical models.

The modular nature of DTMs enables various hardware implementations. For example, each EBM in the chain can be implemented using distinct physical circuitry on the same chip, as shown in Fig. 3 (b). Alternatively, the various EBMs may be split across several communicating chips or implemented by the same hardware, reprogrammed with distinct sets of weights at different times. For any given EBM in the chain, both the data variables x^t, x^{t-1} and the latent variables z^{t-1} are physically embodied in sampling circuits that are connected in a simple way that reflects the structure of Eq. (7). This variable structure is shown schematically in Fig. 3 (c).

To understand the performance of a future hardware device, we developed a GPU simulator of the DTCA and used it to train a DTM on the Fashion-MNIST dataset. We measure the performance of the DTM using FID and utilize a physical model to estimate the energy required to generate new images. These numbers can be compared to conventional algorithm/hardware pairings, such as a VAE running on a GPU; these results are shown in Fig. 1.

The DTM that produced the results shown in Fig. 1 used Boltzmann machine EBMs. Boltzmann machines, also known as Ising models in physics, use binary random variables and are the simplest type of discrete-variable EBM.

Boltzmann machines are hardware efficient because the Gibbs sampling update rule required to sample from them is simple. Boltzmann machines implement energy functions of the form

$$\mathcal{E}(x) = -\beta\Big(\sum_{i\neq j} x_i J_{ij} x_j + \sum_{i=1} h_i x_i\Big), \tag{10}$$

where each $x_i \in \{-1, 1\}$. The Gibbs sampling update rule for sampling from the corresponding EBM is

$$\mathbb{P}\left(X_i[k+1] = +1 \mid X[k] = x\right) = \sigma\Big(2\beta\Big(\sum_{j\neq i} J_{ij}\, x_j + h_i\Big)\Big), \tag{11}$$

which can be evaluated simply using an appropriately biased source of random bits.

Implementing our proposed hardware architecture using Boltzmann machines is particularly simple. A device will consist of nodes connected into a regular grid. Each node represents a single Bernoulli random variable x_i, implemented as a transistor-based sampling circuit (later referred to as RNG). The bias of a sampling circuit (the probability that it produces 1 as opposed to -1) is constrained to be a sigmoidal function of an input voltage, allowing its update probability to be conditioned by the values of its neighbors as given in Eq. (11). This can be implemented using a simple circuit that adds currents such as a resistor network (see Appendix E.1).

Specifically, the EBMs employed in this work were sparse, deep Boltzmann machines comprising $L \times L$ grids of binary variables, where $L = 70$ was used in most cases. Each variable was connected to several (in most cases, 12) of its neighbors following a simple pattern. At random, some of the variables were selected to represent the data x_{t-1}, and the rest were assigned to the latent variables z_{t-1}. Then, an extra node was connected to each data node to implement the coupling to x_t. See Appendix D for further details on the Boltzmann machine architecture.

Due to our chosen connectivity patterns, our Boltzmann machines are bipartite (two-colorable), meaning that nodes can be separated into two blocks, such that no two nodes within the same block are connected to each other. Since each color block can be sampled in parallel, a single iteration of Gibbs sampling corresponds to first sampling one color block conditioned on the other and then swapping their roles. Therefore, the total duration of one full Gibbs iteration (so updating all nodes) takes roughly $2\tau_0$ in wall-clock time, where τ_0 is the decorrelation time of an individual RNG, that is the time it takes for a sampling circuit inside a single node of the Boltzmann machine to equilibrate. It should be noted that τ_0 is set by the intrinsic stochastic dynamics of the subthreshold circuit and its operating point. It is not a freely tunable algorithmic parameter, but can only be modified indirectly through circuit redesign.

Starting from some random initialization, this block-sampling procedure can be repeated for K iterations (where K is longer than the mixing time of the sampler, typically $K \approx 1000$) to draw samples from Eq. (7) for each step in the approximation to the reverse process. For a DTM running on a DTCA chip, the total time required to draw a single sample is therefore proportional to $TK\,\tau_0$, where T is the number of denoising steps (i.e., EBMs that the DTM is comprised of). It is important to distinguish τ_0 from K_{mix}. The former is an entirely

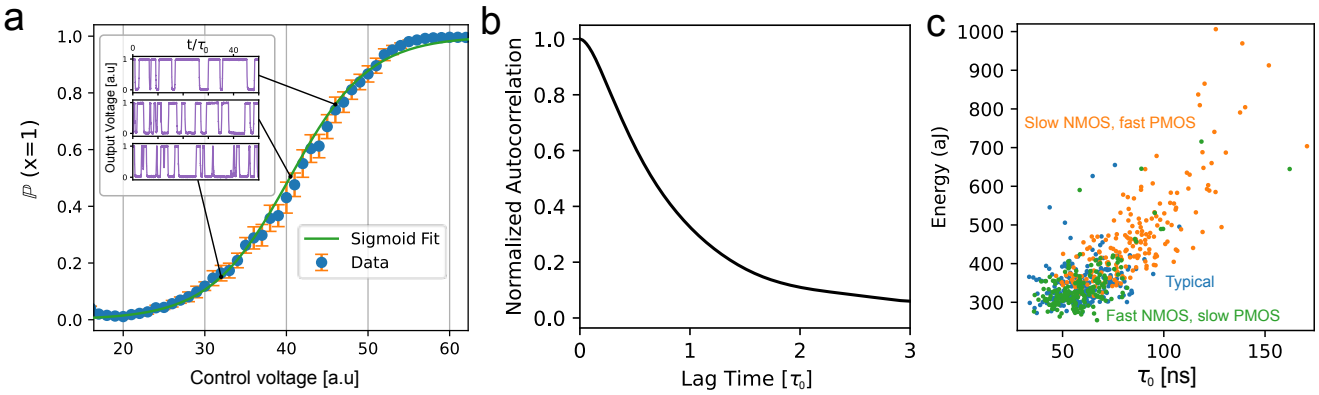

FIG. 4. **A programmable source of random bits. (a)** A laboratory measurement of the operating characteristic of our RNG. The probability of the output voltage signal being in the high state ($x = 1$) can be programmed by varying an input voltage. The relationship between $\mathbb{P}(x = 1)$ and the input voltage is well-approximated by a sigmoid function. The inset shows the output voltage signal as a function of time for different input voltages. **(b)** The autocorrelation function of the RNG at the unbiased point ($\mathbb{P}(x = 1) = 0.5$). The decay is approximately exponential with the rate $\tau_0 \approx 100$ns. **(c)** Estimating the effect of manufacturing variation on RNG performance. Each point in the plot represents the results of a simulation of an RNG circuit with transistor parameters sampled according to a procedure defined by the manufacturer's PDK. Each color represents a different process corner, each for which ~ 200 realizations of the RNG were simulated. The "typical" corner represents a balanced case, whereas the other two are asymmetric corners where the two types of transistors (NMOS and PMOS) are skewed in opposite directions. The slow NMOS and fast PMOS case is worst performing for us due to an asymmetry in our design.

hardware-related parameter, while the latter is the number of Gibbs iterations that the abstract Boltzmann machine (irrespective of hardware implementation) needs to approximately converge to its equilibrium distribution.

To enable a near-term, large-scale realization of the DTCA, we leveraged the shot-noise dynamics of subthreshold transistors [58] to build an RNG that is fast, energy-efficient, and small. Our all-transistor RNG is programmable and has the desired sigmoidal response to a control voltage, as shown by experimental measurements in Fig. 4 (a). The stochastic voltage signal output from the RNG has an approximately exponential autocorrelation function that decays in approximately 100 ns, as illustrated in Fig. 4 (b). As shown in Ref. [58], this time constant is much larger than the lower limit imposed by the correlation time of the noise in our transistors. The RNG could, therefore, be made much faster via an improved design. Appendix K provides further details about our RNG.

In practice, semiconductor manufacturing is never perfectly precise. Each fabrication step introduces small fluctuations in transistor characteristics, producing global shifts across wafers as well as random differences between neighboring devices on the same chip. Because these variations can change how fast or efficiently a circuit operates, it is important to verify that our approach remains reliable under realistic fabrication conditions. A practical advantage of our all-transistor RNG is that detailed and proven foundry-provided models can be used to study the effect of manufacturing variations on our circuit design. In Fig. 4 (c), we use this process development kit (PDK) to study the speed and energy consumption of our RNG as a function of both systematic inter-wafer skews to the transistor parameters (process corners) and the expected variation within a single chip. We find that the RNG works reliably despite these nonidealities, meaning it can be readily scaled to the massive grids required by the DTCA.

The energy estimates given in Fig. 1 for the probabilistic computer were constructed using a physical model of an all-transistor Boltzmann machine Gibbs sampler. The dominant contributions to this model are captured by the formula

$$E = TK_{\text{mix}}L^2E_{\text{cell}}, \tag{12}$$

$$E_{\text{cell}} = E_{\text{rng}} + E_{\text{bias}} + E_{\text{clock}} + E_{\text{comm}}, \tag{13}$$

where E_{rng} comes from the data in Fig. 4 (c). The term E_{bias} is estimated using a physical model of a possible biasing circuit, and E_{clock} and E_{comm} are derived from physical reasoning about the costs of the clock and intercell communications, respectively. K_{mix} is the number of sampling iterations required to satisfactorily mix the chain for inference, which is generally less than the number of iterations used during training. $K_{\text{mix}} = 250$ was used for the DTM (see Appendix E.4), while the mixing time measured in Fig. 2 was used for the MEBM.

This model is approximate, but it captures the underlying physics of a real device and provides a reasonable order-of-magnitude estimate of the actual energy consumption. Generally, given the same transistor process we used for our RNG and some reasonable selections for other free parameters of the model, we can estimate $E_{\text{cell}} \approx 2$ fJ. See Appendix E for an exhaustive derivation of this model.

We use a simple model for the energy consumption of the GPU that underestimates the actual values. We compute the total number of floating-point operations (FLOPs) required to generate a sample from the trained model and divide that by the FLOP/joule specification given by the manufacturer. See Appendix F for further discussion.

IV. TRAINING DTMS

The EBMs used in the experiments presented in Fig. 1 were trained by applying the standard Monte Carlo estimator for the gradients of EBMs [59] to Eq. (4), which yields

$$\nabla_\theta \mathcal{L}_{DN}(\theta) = \sum_{t=1}^{T} \mathbb{E}_{Q(x_{t-1},x_t)} \Big[\mathbb{E}_{P_\theta(z_{t-1}|x_{t-1},x_t)} \left[\nabla_\theta \mathcal{E}^m_{t-1} \right] - \mathbb{E}_{P_\theta(x_{t-1},z_{t-1}|x_t)} \left[\nabla_\theta \mathcal{E}^m_{t-1} \right] \Big]. \tag{14}$$

Notably, each term in the sum over t can be computed independently. To estimate either term in Eq. (14), first, sample tuples (x_{t-1}, x_t) from the forward process $Q(x_{t-1}, x_t)$. Then, for each of these tuples, clamp the reverse process EBM to the sampled values appropriately and use a time average over K iterations of Gibbs sampling to estimate the inner expectation value. Averaging the result over the tuples yields the desired gradient estimate.

It should be noted that the DTCA allows our EBMs to have finite and short mixing times, which enables sufficient sampling iterations to be used to achieve nearly unbiased estimates of the gradient. Unbiased gradient estimates are not possible for MEBMs in most cases due to their long mixing times [60].

A well-trained denoising model generates new examples that resemble the training data by incrementally pulling them out of noise; the outputs of an 8-step denoising model trained on the Fashion-MNIST dataset are shown in Fig. 5 (a). At the final time T, the images are random bits. Structure begins to emerge as the chain progresses, ultimately resulting in clean images at time $t = 0$.

DTMs alleviate the training instability that is fundamental to MEBMs. The parameters of MEBMs are usually initialized using a strategy that results in an easy-to-sample-from energy landscape [61]. For this reason, in the early stages of training, sampling from Eq. (1) is possible, and the gradient estimates produced using Eq. (14) are unbiased. However, as these gradients are followed, the MEBM is reshaped according to the data distribution and begins to become complex and multimodal. This induced multimodality greatly increases the sampling complexity of the distribution, causing samples to deviate from equilibrium. Gradients computed using non-equilibrium samples do not necessarily point in a meaningful direction, which can halt or, in some cases, even reverse the training process.

This instability in MEBMs leads to unpredictable training dynamics that can be sensitive to implementation details. An example of the training dynamics for several different types of models is shown in Fig. 5 (b). The top plot displays the quality of images generated during training, while the bottom plot shows a measure of the sampler's mixing. Image quality is measured using the FID metric, and mixing quality is measured using normalized autocorrelation r_{yy} (see Appendix G). The lower plot in Fig. 5 (b) shows the autocorrelation at a delay equal to the total number of sampling iterations used to estimate the gradients during training. Generally, if r_{yy} is close to 1, gradients were estimated using far-from-equilibrium samples and were likely of low quality. If it is close to zero, the samples should be close to equilibrium and produce high-quality gradient estimates. See Appendix G for further discussion.

The destabilizing effect of non-equilibrium sampling is apparent from the blue curves in Fig. 5 (b). At the beginning of training, both quality and r_{yy} increase, indicating that the multimodality of the data is being imprinted on the model. As training progresses and the energy landscape becomes more rugged, r_{yy} becomes so large that the quality of the gradient starts to decline, resulting in a plateau and, ultimately, a degradation of the model's quality.

Denoising alone significantly stabilizes training. Because the transformation carried out by each layer is simpler, the distribution that the model must learn is less complex and, therefore, easier to sample from. The orange curve in Fig. 5 (b) shows the training dynamics for a typical denoising model. The autocorrelation and performance remain good for much longer than the MEBM.

As training progresses, the DTM eventually becomes unstable, which can be attributed to the development of a complex energy landscape among the latent variables. To combat this, we modify the training procedure to penalize models that mix poorly. We add a term to the loss function that nudges the optimization towards a distribution that is easy to sample from, i.e.,

$$\mathcal{L}^{TC}_t = \mathbb{E}_{Q(x^t)} \left[D\left(\prod_{i=1}^{M} P_\theta(s^{t-1}_i | x^t) \middle\| P_\theta(s^{t-1}|x^t) \right) \right], \tag{15}$$

where $s^{t-1} = (x^{t-1}, z^{t-1})$ and x^{t-1}_i indicates the i^{th} of the M variables in x^{t-1}. This term penalizes the distance between the learned conditional distribution and a factorized distribution with identical marginals and is a form of total correlation penalty [62]. See Appendices H.2 and M for further discussion of this penalty and its relationship to mixing time and sampling effort.

The total loss function is the sum of Eq. (4) and this

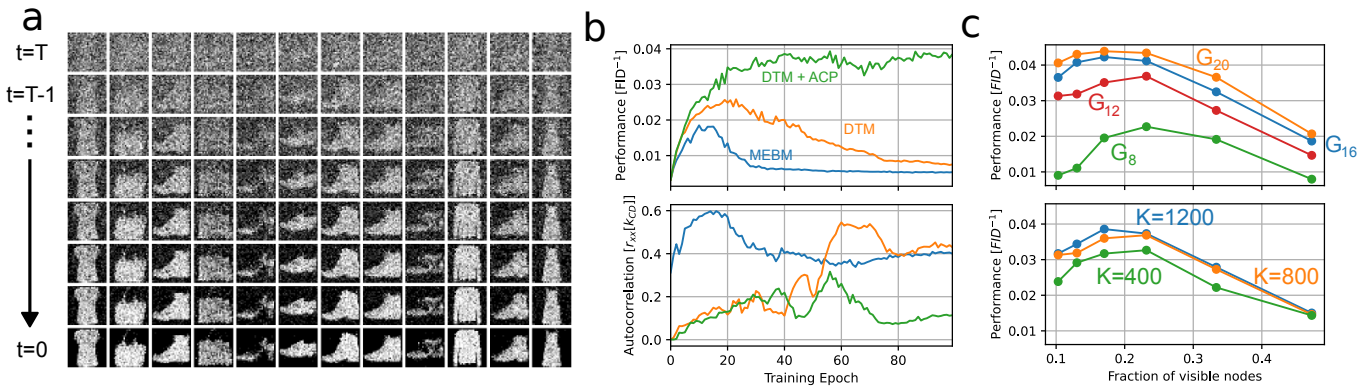

FIG. 5. **Detailed results on the Fashion-MNIST dataset. (a)** Images generated by a denoising model. Here, to achieve better-looking images, several binary variables were combined to represent a single grayscale pixel. The noisiness of the grayscale levels is an artifact of our embedding method; see Appendix I. **(b)** An experiment showing how DTMs are more stable to train than MEBMs. Complementing DTMs with the ACP completely stabilizes training. For the DTMs, the maximum $r_{yy}[K]$ value over all the layers is shown. **(c)** The effect of scaling EBM complexity on DTM performance. The grid size L was modified to change the number of latent variables compared to the (fixed) number of data variables. Generally, EBM layers with more connectivity and longer allowed mixing times can utilize more latent variables and, therefore, achieve higher performance.

total correlation penalty:

$$\mathcal{L} = \mathcal{L}_{DN} + \sum_{t=1}^{T} \lambda_t \mathcal{L}_t^{TC}. \tag{16}$$

The parameters λ_t control the relative strength of the total correlation penalty for each step in the reverse process.

We use an Adaptive Correlation Penalty (ACP) to set the λ_t as large as necessary to keep sampling tractable for each layer. During training, we periodically measure the autocorrelations of each learned conditional at a delay equal to the number of sampling iterations used during gradient estimation. If the autocorrelation for the j^{th} layer is close to zero, λ_j is decreased, and vice versa.

Our closed-loop control of the correlation penalty strengths is crucial, allowing us to maximize the expressivity of the EBMs while maintaining stable training. The green curves in Fig. 5 (b) show an example of training dynamics under this closed-loop control policy. Model quality increases monotonically, and the autocorrelation stays small throughout training. This closed-loop control of the correlation penalty was employed during the training of most models used to produce the results in this article, including those shown in Fig. 1. Further analysis of the ACP and its effect on mixing is given in Appendices H.2 and M.

Generally, the performance of DTMs improves as their size increases. As shown in Fig. 1, increasing the depth of the DTM from 2 to 8 substantially improves the quality of generated images. As shown in Fig. 5 (c), increasing the width, degree, and allowed mixing time of the EBMs in the chain also generally improves performance.

However, some subtleties prevent this specific EBM topology from being scaled indefinitely. The top plot in Fig. 5 (c) shows that scaling the number of latent variables (with fixed allowed mixing time) only improves performance if the connectivity of the graph is also scaled; otherwise, performance can decrease. This dependence makes sense, as increasing the number of latent variables in this way increases the depth of the Boltzmann machine, which is known to make sampling more difficult. Beyond a certain point, increasing the model's ability to express complex energy landscapes may render it unable to learn, given the allowed mixing time of $K \approx 1000$. This same effect is shown in the bottom plot of Fig. 5 (c), which demonstrates that larger values of K are required to support wider models while holding connectivity constant. Further study of how performance scales with sampling time K and the number of denoising steps T is provided in Appendix M.

In general, it would be naive to expect that a hardware-efficient EBM topology can be scaled in isolation to model arbitrarily complex datasets. For example, there is no good reason why a connectivity pattern that is convenient from a wire-routing perspective would also be well suited to represent the correlation structure of a complex real-world dataset.

V. HYBRID THERMODYNAMIC-DETERMINISTIC MACHINE LEARNING

The core doctrine of modern machine learning is the relentless scaling of models as a means of solving ever-harder problems. Models that utilize probabilistic computers may be similarly scaled to enhance their capabilities beyond the relatively simple dataset considered in this work so far.

However, we hypothesize that the correct way to scale probabilistic machine learning hardware systems is

not in isolation but rather as a component in a larger *hybrid thermodynamic-deterministic machine learning* (HTDML) system. Such a hybrid system integrates probabilistic hardware with more traditional machine learning accelerators. This hybrid approach is particularly important for moving beyond binarized grayscale datasets such as Fashion-MNIST toward richer, higher-dimensional data.

A hybrid approach is sensible because there is no *a priori* reason to believe that a probabilistic computer should handle every part of a machine learning problem, and sometimes a deterministic processor is likely a better tool for the job.

The goal of HTDML is to design practical machine learning systems that minimize the energy used to achieve desired modeling fidelity on a particular task. This efficiency will be achieved through a cross-disciplinary effort that eschews the software/hardware abstraction barrier to design computers that respect physical constraints and leverage each type of hardware where it is most effective.

For example, more rigorous methods of embedding data into hardware EBMs will need to be developed to go beyond the relatively simple datasets considered here. Indeed, binarization is not viable in general, and embedding into richer types of variables (such as categorical) at the probabilistic hardware level is not particularly efficient or principled. Instead, one may try a hybrid (HTDML) approach where a small neural network (the deterministic part) learns to embed the data into a binary latent space and a DTM (the thermodynamic part) can then be trained inside this binary space. A similar approach has been very beneficial in the context of diffusion models [55], to the point that now many state-of-the-art diffusion models operate in the latent space of a pre-trained VAE.

We show the results of a quick initial attempt at such hybrid models in Fig. 6. Here, we train a small neural network to embed the CIFAR-10 dataset [63] into a binary DTM. The embedding network was trained using an autoencoder loss to binarize the data, which was then used to train a DTM. The decoder of the embedding network was then trained further using a GAN objective to increase the quality of the generated images. This training procedure is described in further detail in Appendix J. Example CIFAR-10 images generated by our model can be found in Appendix N.

Despite the overhead of the embedding neural network, this primitive hybrid model is efficient. As shown in the figure, to achieve equivalent performance with a purely NN-based GAN, the deterministic part of that GAN would need to have roughly ten times more parameters than the deterministic part of our hybrid model. Informally, one might say that therefore the DTM holds about 90% of the expressivity of our model while the neural network accounts for only about 10% of the parameters.

We should again stress that this is just a naive first attempt at a hybrid model, and we believe that further research into hybrid thermodynamic–deterministic models can lead to even more significant efficiency gains and scale this approach to larger tasks. For example, one major flaw of the embedding procedure employed in Fig. 6 is that the autoencoder and DTM are not jointly trained, which means that the embedding learned by the autoencoder may not be well-suited to the way information can flow in the DTM, given its limited connectivity. Finding a good way to jointly train them seems like a promising future research direction.

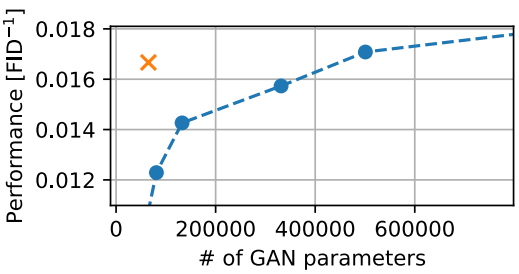

FIG. 6. **Embedding data into a DTM using a neural network.** Here, we show the results of using a simple embedding model in combination with a DTM. The DTM is trained to generate CIFAR-10 images and achieves performance parity with a traditional GAN using a $\sim 10\times$ smaller deterministic neural network in terms of parameter count.

VI. CONCLUSION

In this paper, we have given an early view of how one might design generative machine learning algorithms and hardware jointly for thermodynamic probabilistic computers. At the modeling level, we introduced Denoising Thermodynamic Models (DTMs), which repurpose hardware EBMs as denoising steps rather than monolithic models of the data distribution and thereby avoid the mixing–expressivity tradeoff that limits monolithic EBMs. At the hardware level, we proposed the Denoising Thermodynamic Computer Architecture (DTCA), which implements DTMs using sparse, locally connected Boltzmann machines driven by an all-transistor RNG with a tunable bias and short autocorrelation time. By combining these ingredients with an adaptive correlation-penalty training procedure, we estimate that a DTCA device could match the performance of GPU-based generative models on binarized Fashion-MNIST while using roughly four orders of magnitude less energy per generated sample.

It is important to note that the experiments presented in this paper were done using simulations of thermodynamic hardware, running on a classical computer, which substantially limited the maximum size of the thermodynamic model we could represent. Specifically, the largest DTM shown in Fig. 1 uses only around 50,000 nodes in

total, which is much smaller than what we expect the capacity of early DTCA chips will be. In fact, based on the size of our RNG, it can be estimated that $\sim 10^6$ sampling cells could be fit into a 6×6 µm chip (see Appendix K). Furthermore, measurements of our RNG suggest that early DTCA chips will be about two to three orders of magnitude faster than the simulations used in this article. As shown in fig. 17, using more Gibbs steps during training and inference as well as using more denoising steps both increase model performance (albeit with diminishing returns). Furthermore, as shown in fig. 5 (c, bottom), as the model gets larger, increasing the number of Gibbs steps has a larger impact on performance. Realizing large-scale probabilistic computers using advanced transistor processes [64–66] will therefore both increase the capabilities of the DTM algorithm presented here and substantially speed up future research into other thermodynamic algorithms. The present work should thus be viewed as establishing a qualitative scaling picture and design template rather than as an optimized end-point.

Given the gap between the size of our models and the capabilities of a potential hardware device, a natural question to study is how these probabilistic models can be scaled beyond the obvious approaches considered here. This scaling likely corresponds to developing architectures that fuse multiple EBMs to implement each step in the reverse process. One possible approach is to construct software-defined graphical models of EBMs that enable non-local information routing, which could alleviate some of the issues associated with a fixed and local interaction structure. Such research would go hand-in-hand with HTDML, since classical neural networks could be used to learn an embedding that hierarchically distributes the generative task between several EBMs in a way similar to nVAE [67] or StyleGAN [68]. Our preliminary CIFAR-10 experiments already show that pairing a relatively small deterministic front-end with a DTM back-end can offload most of the generative burden onto the thermodynamic component.

To address the current unavailability of probabilistic hardware, we have open-sourced a software library [69] that enables XLA-accelerated [70] simulation of hardware EBMs. This library is written in JAX [71] and automates the complex slicing operations that enable hardware EBM sampling. We also provide additional code that wraps this library to implement the specific experiments presented in this article [72]. By making these tools available, we aim to lower the barrier for others to explore alternative thermodynamic algorithms, hardware topologies, and hybrid architectures, and to stress-test and refine the DTCA design in a broader range of applications.

In short, DTMs and the DTCA provide a concrete path toward generative models and hardware that are co-designed for thermodynamic probabilistic computation, rather than retrofitted to existing digital accelerators.

VII. DECLARATIONS

A. Data Availability and Code Availabilty

The code which reproduces our results and data is available at github.com/pschilliOrange/dtm-replication.

B. Acknowledgements

This research was funded by Extropic Corp.

C. Author Contributions

AJ and TM contributed equally, with TM doing most of the writing and energy estimation work, whereas AJ contributed most to the algorithms for training and evaluating the models presented. OL evaluated the performance of competitive models that we were comparing against. AG helped design some of the mentioned integrated circuits. PS helped with some experiments, and formatted the research code (which was published on GitHub) to be easy to understand and use. IC advised TM at the outset of this project. GV provided suggestions at the idea stage.

D. Competing Interests

All authors are part of Extropic corp., which produces and aims to sell products using some of the technologies described in the paper.

[1] A. A. Chien, *GenAI: Giga$$$, TeraWatt-Hours, and GigaTons of CO2*, Commun. ACM **66**, 5 (2023).

[2] D. D. Stine, *The Manhattan Project, the Apollo Program, and Federal Energy Technology R&D Programs: A Comparative Analysis*, Report RL34645 (Congressional Research Service, Washington, D.C., 2009).

[3] J. Aljbour, T. Wilson, and P. Patel, *Powering Intelligence: Analyzing Artificial Intelligence and Data Center Energy Consumption*, EPRI White Paper no. 3002028905 (2024).

[4] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, *Competition-level code generation with AlphaCode*, Science **378**, 1092 (2022).

[5] D. M. Katz, M. J. Bommarito, S. Gao, and P. Arredondo, *GPT-4 passes the bar exam*, Philos. Trans. R. Soc. A **382**, 20230254 (2024).

[6] H. Nori, N. King, S. M. McKinney, D. Carignan, and E. Horvitz, *Capabilities of gpt-4 on medical challenge problems*, arXiv [cs.CL] (2023).

[7] S. Noy and W. Zhang, *Experimental evidence on the productivity effects of generative artificial intelligence*, Science **381**, 187 (2023).

[8] E. Brynjolfsson, D. Li, and L. Raymond, *Generative AI at work*, Q. J. Econ. 10.1093/qje/qjae044 (2025).

[9] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, *The impact of AI on developer productivity: Evidence from GitHub Copilot*, arXiv [cs.SE] (2023).

[10] A. Bick, A. Blandin, and D. J. Deming, The rapid adoption of generative ai, Tech. Rep. (National Bureau of Economic Research, 2024).

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, Attention is all you need, in *Advances in Neural Information Processing Systems*, Vol. 30, edited by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Curran Associates, Inc., 2017).

[12] A. Coates, B. Huval, T. Wang, D. J. Wu, A. Y. Ng, and B. Catanzaro, Deep learning with cots hpc systems, in *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28*, ICML'13 (JMLR.org, 2013) p. III–1337–III–1345.

[13] K. Chellapilla, S. Puri, and P. Simard, High Performance Convolutional Neural Networks for Document Processing, in *Tenth International Workshop on Frontiers in Handwriting Recognition*, edited by G. Lorette, Université de Rennes 1 (Suvisoft, La Baule (France), 2006).

[14] S. Hooker, The hardware lottery, Commun. ACM **64**, 58–65 (2021).

[15] S. Ambrogio, P. Narayanan, A. Okazaki, A. Fasoli, C. Mackin, K. Hosokawa, A. Nomura, T. Yasuda, A. Chen, A. Friz, *et al.*, *An analog-AI chip for energy-efficient speech recognition and transcription*, Nature **620**, 768 (2023).

[16] S. Bandyopadhyay, A. Sludds, S. Krastanov, R. Hamerly, N. Harris, D. Bunandar, M. Streshinsky, M. Hochberg, and D. Englund, *Single-chip photonic deep neural network with forward-only training*, Nat. Photon. **18**, 1335 (2024).

[17] H. A. Gonzalez, J. Huang, F. Kelber, K. K. Nazeer, T. Langer, C. Liu, M. Lohrmann, A. Rostami, M. Schone, B. Vogginger, *et al.*, *SpiNNaker2: A large-scale neuromorphic system for event-based and asynchronous machine learning*, arXiv [cs.ET] (2024).

[18] S. B. Shrestha, J. Timcheck, P. Frady, L. Campos-Macias, and M. Davies, *Efficient Video and Audio Processing with Loihi 2*, in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (2024) pp. 13481–13485.

[19] Y. Song and S. Ermon, Generative modeling by estimating gradients of the data distribution, in *Advances in Neural Information Processing Systems*, Vol. 32, edited by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Curran Associates, Inc., 2019).

[20] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, *Planning with Diffusion for Flexible Behavior Synthesis*, in *International Conference on Machine Learning* (PMLR, 2022) pp. 9902–9915.

[21] T. Conte, E. DeBenedictis, N. Ganesh, T. Hylton, J. P. Strachan, R. S. Williams, A. Alemi, L. Altenberg, G. Crooks, J. Crutchfield, *et al.*, *Thermodynamic computing*, arXiv [cs.CY] (2019).

[22] H. Xiao, K. Rasul, and R. Vollgraf, *Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms*, arXiv [cs.LG] (2017).

[23] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, GANs trained by a two time-scale update rule converge to a local Nash equilibrium, in *Advances in Neural Information Processing Systems*, Vol. 30, edited by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Curran Associates, Inc., 2017).

[24] D. P. Kingma and M. Welling, *Auto-Encoding Variational Bayes* (2022).

[25] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, Generative adversarial nets, in *Advances in Neural Information Processing Systems*, Vol. 27, edited by Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger (Curran Associates, Inc., 2014).

[26] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*, in *Proceedings of the 32nd International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 37, edited by F. Bach and D. Blei (PMLR, Lille, France, 2015) pp. 2256–2265.

[27] N. S. Singh, K. Kobayashi, Q. Cao, K. Selcuk, T. Hu, S. Niazi, N. A. Aadit, S. Kanai, H. Ohno, S. Fukami, *et al.*, *CMOS plus stochastic nanomagnets enabling heterogeneous computers for probabilistic inference and learning*, Nat. Commun. **15**, 2685 (2024).

[28] C. Pratt, K. Ray, and J. Crutchfield, *Dynamical Computing on the Nanoscale: Superconducting Circuits for Thermodynamically-Efficient Classical Information Processing* (2023).

[29] G. Wimsatt, O.-P. Saira, A. B. Boyd, M. H. Matheny, S. Han, M. L. Roukes, and J. P. Crutchfield, *Harnessing fluctuations in thermodynamic computing via time-reversal symmetries*, Phys. Rev. Res. **3**, 033115 (2021).

[30] S. H. Adachi and M. P. Henderson, *Application of Quantum Annealing to Training of Deep Neural Networks* (2015).

[31] B. Sutton, K. Y. Camsari, B. Behin-Aein, and S. Datta, *Intrinsic optimization using stochastic nanomagnets*, Sci. Rep. **7**, 44370 (2017).

[32] R. Faria, K. Y. Camsari, and S. Datta, *Low-Barrier Nanomagnets as p-Bits for Spin Logic*, IEEE Magn. Lett. **8**, 1 (2017).

[33] S. Niazi, S. Chowdhury, N. A. Aadit, M. Mohseni, Y. Qin, and K. Y. Camsari, *Training deep Boltzmann networks with sparse Ising machines*, Nat. Electron. **7**, 610 (2024).

[34] W. A. Borders, A. Z. Pervaiz, S. Fukami, K. Y. Camsari, H. Ohno, and S. Datta, *Integer factorization using stochastic magnetic tunnel junctions*, Nature **573**, 390 (2019).

[35] N. S. Singh, K. Kobayashi, Q. Cao, K. Selcuk, T. Hu, S. Niazi, N. A. Aadit, S. Kanai, H. Ohno, S. Fukami, *et al.*, *CMOS plus stochastic nanomagnets enabling heterogeneous computers for probabilistic inference and*

learning, Nat. Commun. **15**, 2685 (2024).
[36] M. M. H. Sajeeb, N. A. Aadit, S. Chowdhury, T. Wu, C. Smith, D. Chinmay, A. Raut, K. Y. Camsari, C. Delacour, and T. Srimani, Scalable connectivity for Ising machines: Dense to sparse, Phys. Rev. Appl. **24**, 014005 (2025).
[37] Y. Du and I. Mordatch, Implicit generation and modeling with energy based models, in *Advances in Neural Information Processing Systems*, Vol. 32, edited by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Curran Associates, Inc., 2019).
[38] W. Lee, H. Kim, H. Jung, Y. Choi, J. Jeon, and C. Kim, *Correlation free large-scale probabilistic computing using a true-random chaotic oscillator p-bit*, Sci. Rep. **15**, 8018 (2025).
[39] M. Horodynski, C. Roques-Carmes, Y. Salamin, S. Choi, J. Sloan, D. Luo, and M. Soljačić, *Stochastic logic in biased coupled photonic probabilistic bits*, Commun. Phys. **8**, 31 (2025).
[40] M. A. Abeed and S. Bandyopadhyay, *Low Energy Barrier Nanomagnet Design for Binary Stochastic Neurons: Design Challenges for Real Nanomagnets With Fabrication Defects*, IEEE Magn. Lett. **10**, 1 (2019).
[41] M. A. Abeed and S. Bandyopadhyay, *Sensitivity of the Power Spectra of Thermal Magnetization Fluctuations in Low Barrier Nanomagnets Proposed for Stochastic Computing to In-Plane Barrier Height Variations and Structural Defects*, SPIN **10**, 2050001 (2020).
[42] J. L. Drobitch and S. Bandyopadhyay, *Reliability and Scalability of p-Bits Implemented With Low Energy Barrier Nanomagnets*, IEEE Magn. Lett. **10**, 1 (2019).
[43] M. I. Jordan and T. M. Mitchell, *Machine learning: Trends, perspectives, and prospects*, Science **349**, 255 (2015).
[44] Z. Ghahramani, *Probabilistic machine learning and artificial intelligence*, Nature **521**, 452 (2015).
[45] G. Hinton, Boltzmann Machines: Constraint Satisfaction Networks that Learn, Carnegie-Mellon University. Department of Computer Science (Carnegie-Mellon University, Department of Computer Science, 1984).
[46] A. P. Dempster, N. M. Laird, and D. B. Rubin, *Maximum likelihood from incomplete data via the EM algorithm*, J. R. Stat. Soc. Ser. B (Methodol.) **39**, 1 (1977).
[47] C. M. Bishop, Mixture density networks (1994).
[48] K. P. Murphy, Probabilistic Machine Learning: Advanced Topics (MIT Press, 2023) p. 499.
[49] D. Carbone, M. Hua, S. Coste, and E. Vanden-Eijnden, *Efficient training of energy-based models using Jarzynski equality*, Adv. Neural Inf. Process. Syst. **36**, 52583 (2023).
[50] G. Desjardins, A. Courville, Y. Bengio, P. Vincent, O. Delalleau, *et al.*, *Parallel tempering for training of restricted Boltzmann machines*, in *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (MIT Press Cambridge, MA, 2010) pp. 145–152.
[51] J. Ho, C. Saharia, W. Chan, D. J. Fleet, M. Norouzi, and T. Salimans, *Cascaded diffusion models for high fidelity image generation*, J. Mach. Learn. Res. **23**, 1 (2022).
[52] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, *High-resolution image synthesis with latent diffusion models*, in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2022) pp. 10684–10695.
[53] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, J. Ho, D. J. Fleet, and M. Norouzi, Photorealistic text-to-image diffusion models with deep language understanding, in *Advances in Neural Information Processing Systems*, Vol. 35, edited by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Curran Associates, Inc., 2022) pp. 36479–36494.
[54] R. Gao, Y. Song, B. Poole, Y. N. Wu, and D. P. Kingma, *Learning Energy-Based Models by Diffusion Recovery Likelihood*, arXiv [cs.LG] (2021).
[55] P. Yu, S. Xie, X. Ma, B. Jia, B. Pang, R. Gao, Y. Zhu, S.-C. Zhu, and Y. N. Wu, *Latent Diffusion Energy-Based Model for Interpretable Text Modelling*, in *International Conference on Machine Learning* (PMLR, 2022) pp. 25702–25720.
[56] M. Xu, T. Geffner, K. Kreis, W. Nie, Y. Xu, J. Leskovec, S. Ermon, and A. Vahdat, *Energy-based diffusion language models for text generation*, arXiv [cs.CL] (2024).
[57] Y. Zhu, J. Xie, Y. N. Wu, and R. Gao, *Learning Energy-Based Models by Cooperative Diffusion Recovery Likelihood*, in *The Twelfth International Conference on Learning Representations* (2021).
[58] N. Freitas, G. Massarelli, J. Rothschild, D. Keane, E. Dawe, S. Hwang, A. Garlapati, and T. McCourt, *Taming non-equilibrium thermal fluctuations in subthreshold CMOS circuits*, Phys. Rev. Lett. (2025), Submitted.
[59] Y. Song and D. P. Kingma, *How to train your energy-based models*, arXiv [cs.LG] (2021).
[60] M. A. Carreira-Perpinan and G. Hinton, *On contrastive divergence learning*, in *International workshop on artificial intelligence and statistics* (PMLR, 2005) pp. 33–40.
[61] G. E. Hinton, *A practical guide to training restricted Boltzmann machines*, in *Neural Networks: Tricks of the Trade: Second Edition* (Springer, 2012) pp. 599–619.
[62] R. T. Chen, X. Li, R. B. Grosse, and D. K. Duvenaud, *Isolating sources of disentanglement in variational autoencoders*, Adv. Neural Inf. Process. Syst. **31** (2018).
[63] A. Krizhevsky and G. Hinton, *Learning multiple layers of features from tiny images*, Tech. Rep. 0 (University of Toronto, Toronto, Ontario, 2009).
[64] T. Sekigawa and Y. Hayashi, *Calculated threshold-voltage characteristics of an XMOS transistor having an additional bottom gate*, Solid-State Electron. **27**, 827 (1984).
[65] S.-Y. Wu, C.-H. Chang, M. Chiang, C. Lin, J. Liaw, J. Cheng, J. Yeh, H. Chen, S. Chang, K. Lai, *et al.*, *A 3nm CMOS FinFlex™ platform technology with enhanced power efficiency and performance for mobile SoC and high performance computing applications*, in *2022 International Electron Devices Meeting (IEDM)* (IEEE, 2022) pp. 27–5.
[66] J. Liu, S. Mukhopadhyay, A. Kundu, S. Chen, H. Wang, D. Huang, J. Lee, M. Wang, R. Lu, S. Lin, Y. Chen, H. Shang, P. Wang, H. Lin, G. Yeap, and J. He, *A Reliability Enhanced 5nm CMOS Technology Featuring 5th Generation FinFET with Fully-Developed EUV and High Mobility Channel for Mobile SoC and High Performance Computing Application*, in *2020 IEEE International Electron Devices Meeting (IEDM)* (2020) pp. 9.2.1–9.2.4.
[67] A. Vahdat and J. Kautz, NVAE: A deep hierarchical variational autoencoder (2021), arXiv:2007.03898 [stat.ML].
[68] T. Karras, S. Laine, and T. Aila, A style-based generator architecture for generative adversarial networks (2019),

arXiv:1812.04948 [cs.NE].
[69] Extropic, *thrml: Thermodynamic Hypergraphical Model Library* (2025).
[70] A. Sabne, *XLA : Compiling Machine Learning for Peak Performance* (2020).
[71] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, *JAX: composable transformations of Python+NumPy programs* (2018).
[72] *github.com/pschilliOrange/dtm-replication*.

Appendix A: List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
ACP	Adaptive Correlation Penalty
DTM	Denoising Thermodynamic Model
DTCA	Denoising Thermodynamic Computer Architecture
EBM	Energy-Based Model
MEBM	Monolithic Energy-Based Model
FID	Fréchet Inception Distance
FLOP	Floating-Point Operation
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
HTDML	Hybrid Thermodynamic–Deterministic Machine Learning
LLM	Large Language Model
NN	Neural Network
PDK	Process Development Kit
RNG	Random-Number Generator
VAE	Variational Autoencoder
DDPM	Denoising Diffusion Probabilistic Model

TABLE I. Acronyms used throughout this work.

Appendix B: Denoising Diffusion Models

Denoising diffusion models try to learn to time-reverse a random process that converts data into simple noise. Here, we will review some details on how these models work to support the analysis in the main text.

1. Forward Processes

The forward process is a random process that is used to convert the data distribution into noise. This conversion into noise is achieved through a stochastic differential equation in the continuous-variable case and a Markov jump process in the discrete case.

a. Continuous Variables

In the continuous case, the typical choice of forward process is the Itô diffusion,

$$dX(t) = -X(t)dt + \sqrt{2}\sigma dW \tag{B1}$$

where $X(t)$ is a length N vector representing the state variable at time t, σ is a constant, and dW is a length N vector of independent Wiener processes.

The transition kernel for a random process defines how the probability distribution evolves in time,

$$Q_{t|0}(x'|x) = \mathbb{P}(X(t) = x'|X(0) = x) \tag{B2}$$

For the case of Eq. (B1) the transition kernel is,

$$Q_{t+s|s}(x'|x) \propto e^{-\frac{1}{2}(x'-\mu)^T\Sigma^{-1}(x'-\mu)} \tag{B3}$$

$$\mu = e^{-t}x \tag{B4}$$

$$\Sigma = \sigma^2 I \left(1 - e^{-2t}\right) \tag{B5}$$

this solution can be verified by direct substitution into the corresponding Fokker-Planck equation. In the limit of infinite time, $\mu \to 0$ and $\Sigma \to \sigma^2 I$. Therefore, the stationary distribution of this process is zero-mean Gaussian noise with a standard deviation of σ.

b. Discrete Variables

The stochastic dynamics of some discrete variable X may be described by the Markov jump process,

$$\frac{dQ_t}{dt} = \mathcal{L} Q_t \tag{B6}$$

where $\mathcal{L}$ is the generator of the dynamics, which is an $M \times M$ matrix that stores the transition rates between the various states. Q_t is a length M vector that assigns a probability to each possible state X may take at time t.

The transition rate from the i^{th} state to the j^{th} state is given by the matrix element $\mathcal{L}[j,i]$, which here takes the particular form,

$$\mathcal{L}[j,i] = \gamma\left(-(M-1)\delta_{j,i} + (1-\delta_{j,i})\right) \tag{B7}$$

where δ is used to indicate the Kronecker delta function. Eq. (B7) describes a random process where the probability per unit time to jump between any two states is γ.

Since Eq. (B6) is linear, the dynamics of Q_t can be understood entirely via the eigenvalues and eigenvectors of $\mathcal{L}$,

$$\mathcal{L} v_k = \lambda_k v_k \tag{B8}$$

Note that by symmetry of $\mathcal{L}$, we do not distinguish between right and left eigenvectors.

One eigenvector-eigenvalue pair $(v_0, \lambda_0 = 0)$ corresponds to the unique stationary state of $\mathcal{L}$, with all entries of v_0 being equal to some constant (if normalized, then $v_0[j] = \frac{1}{M}$ for all j). The long-time dynamics of this MJP transform any initial distribution to a uniform distribution over all states.

The remaining eigenvectors are decaying modes associated with negative eigenvalues. These additional $M-1$ eigenvectors take the form,

$$v_j[i] = -\delta_{i,0} + \delta_{i,j} \tag{B9}$$

$$\lambda_j = -\gamma M \tag{B10}$$

where Eq. (B9) and Eq. (B10) are valid for $j \in [1, M-1]$. Therefore, all solutions to this MJP decay exponentially to the uniform distribution with rate γM.

The time-evolution of Q is given by the matrix exponential,

$$Q_t = e^{\mathcal{L}t} Q_0. \tag{B11}$$

This matrix exponential is evaluated by diagonalizing $\mathcal{L}$,

$$e^{\mathcal{L}t} = P e^{Dt} P^{-1} \tag{B12}$$

where the columns of P are the M eigenvectors v_k and D is a diagonal matrix of the eigenvalues λ_k.

Using the solution for the eigenvalues and eigenvectors found above, we can solve for the matrix elements of $e^{\mathcal{L}t}$,

$$e^{\mathcal{L}t}[j,i] = \delta_{i,j}\left(\frac{1 + (M-1)e^{-\gamma M t}}{M}\right) + (1-\delta_{i,j})\left(\frac{1 - e^{-\gamma M t}}{M}\right) \tag{B13}$$

Using this solution, we can deduce an exponential form for the matrix elements of $e^{\mathcal{L}t}$,

$$e^{\mathcal{L}t}[j,i] = \frac{1}{Z(t)} e^{\Gamma(t)\delta_{i,j}} \tag{B14}$$

$$\Gamma(t) = \ln\left(\frac{1 + (M-1)e^{-\gamma t}}{1 - e^{-\gamma t}}\right) \tag{B15}$$

$$Z(t) = \frac{M}{1 - e^{-\gamma t}} \tag{B16}$$

Now consider a process in which each element of the vector of N discrete variables X undergoes the dynamics described by Eq. (B6) independently. In that case, the differential equation describing the dynamics of the joint distribution Q_t is,

$$\frac{dQ_t}{dt} = \sum_{k=1}^{N} \left(I_1 \otimes \cdots \otimes \mathcal{L}_k \otimes \ldots I_N\right) Q_t \tag{B17}$$

where I_j indicates the identity operator and $\mathcal{L}_j$ the operator from Eq. (B7) acting on the subspace of the j^{th} discrete variable.

The Kronecker product of the matrix exponentials gives the time-evolution of the joint distribution,

$$e^{\mathcal{L}t} = \bigotimes_{k=1}^{N} e^{\mathcal{L}_k t} \tag{B18}$$

with the matrix elements,

$$e^{\mathcal{L}t}[j, i] = \prod_{k=1}^{N} e^{\mathcal{L}_k t}[j_k, i_k] \tag{B19}$$

where j and i are now vectors with N elements, indexed as i_k or j_k respectively.

Using Eqs. (B14) - (B16), we can find an exponential form for the joint process transition kernel (as defined in Eq. (B2)),

$$Q_{t|0}(x'|x) = \frac{1}{Z(t)} \exp\left(\sum_{k=1}^{N} \Gamma_k(t)\delta_{x'[k],x[k]}\right) \tag{B20}$$

$$Z(t) = \prod_{k=1}^{N} Z_k(t) \tag{B21}$$

$\Gamma_k(t)$ and $Z_k(t)$ are as given in Eqs. (B15) and (B16), with each dimension potentially having it's own transition rate γ_k and number of categories M_k.

2. Reverse Processes

In general, a random process for some variable X can be reversed using Bayes' rule,

$$Q_{t|t+\Delta t}(x'|x) = \frac{Q_{t+\Delta t|t}(x|x')Q_t(x')}{Q_{t+\Delta t}(x)} \tag{B22}$$

where the conditionals are as defined in Eq. (B2), and the marginals are,

$$Q_t(x) = \mathbb{P}\left(X(t) = x\right) \tag{B23}$$

A differential equation that describes the reverse process can be found by analyzing Eq. (B22) in the infinitesimal time limit. Specifically, defining a reversed time $t = T - s$ given some arbitrary endpoint T and expanding Eq. (B22) in Δs,

$$Q_{T-s|T-(s-\Delta s)}(x'|x) \approx \delta_{x,x'} + \Delta s\, \mathcal{L}_{\text{rev}}(x', x) \tag{B24}$$

where $\mathcal{L}_{\text{rev}}$ is the generator of the reverse process,

$$\mathcal{L}_{\text{rev}}(x', x) = \frac{Q_{T-s}(x')}{Q_{T-s}(x)} \lim_{\Delta s \to 0} \left[\frac{d}{d\Delta s} Q_{T-(s-\Delta s)|T-s}(x|x') \right] + \delta_{x,x'} \left(\frac{1}{Q_{T-s}(x)} \frac{dQ_{T-s}(x)}{ds} \right) \tag{B25}$$

here, $\delta_{x,x'}$ is used to indicate the Dirac delta function in the continuous case and the Kronecker delta in the discrete case.

If the dynamics of Q are linear and generated by $\mathcal{L}$ like Eq. (B6), we can simplify,

$$\lim_{\Delta s \to 0} \left[\frac{d}{d\Delta s} Q_{T-(s-\Delta s)|T-s}(x|x') \right] = \mathcal{L}(x, x') \tag{B26}$$

In this case, we can re-write Eq. (B25) in the operator form,

$$\mathcal{L}_{\text{rev}} = Q\mathcal{L}^\dagger Q^{-1} + Q^{-1} \frac{dQ}{ds} \tag{B27}$$

where $\mathcal{L}^\dagger$ is the adjoint operator to $\mathcal{L}$. For continuous variables, the adjoint operator is defined as,

$$\int \psi_2 \mathcal{L} \psi_1 dx = \int \psi_1 \mathcal{L}^\dagger \psi_2 dx \tag{B28}$$

for any test functions ψ_1 and ψ_2. For discrete variables, $\mathcal{L}^\dagger = \mathcal{L}^T$.

a. Continuous variables

In the case that the forward process is an Itô diffusion, $\mathcal{L}$ is the generator for the corresponding Fokker-Planck equation,

$$\mathcal{L} = -\sum_i \frac{\partial}{\partial x_i} f_i(x, t) + \frac{1}{2} \sum_{i,j} \frac{\partial}{\partial x_i} \frac{\partial}{\partial x_j} D_{ij}(t) \tag{B29}$$

where D is a symmetric matrix, $D_{ij} = D_{ji}$ that does not depend on x.

Using Eq. (B28) and integration by parts, it can be shown that the adjoint operator is,

$$\mathcal{L}^\dagger = \sum_i f_i \frac{\partial}{\partial x_i} + \frac{1}{2} \sum_{i,j} D_{ij} \frac{\partial}{\partial x_i} \frac{\partial}{\partial x_i} \tag{B30}$$

By directly substituting Eq. (B30) into Eq. (B27) and simplifying, $\mathcal{L}_{\text{rev}}$ can be reduced to,

$$\mathcal{L}_{\text{rev}} = \sum_i \frac{\partial}{\partial x_i} g_i + \frac{1}{2} \sum_{i,j} \frac{\partial}{\partial x_i} \frac{\partial}{\partial x_j} D_{ij} \tag{B31}$$

with the drift vector g,

$$g_i(x, t) = f_i(x, t) - \frac{1}{Q_t(x)} \sum_j \frac{\partial}{\partial x_j} \left[D_{ij}(x, t) Q_t(x) \right] \tag{B32}$$

If Δt is chosen to be sufficiently small, Eq. (B32) can be linearized and the transition kernel is Gaussian,

$$Q_{t|t+\Delta t}(x'|x) \propto \exp\left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \tag{B33}$$

$$\mu = x + \Delta t \, g_i(x, t) \tag{B34}$$

$$\Sigma = \Delta t \, D(t) \tag{B35}$$

Therefore, one can build a continuous diffusion model with arbitrary approximation power by working in the small Δt limit and approximating the reverse process using a Gaussian distribution with a neural network defining the mean vector [1, 2].

b. *Discrete variables*

In a discrete diffusion model, $\mathcal{L}$ is given by Eq. (B17). This tensor product form for $\mathcal{L}$ guarantees that $\mathcal{L}(x', x) = 0$ for any vectors x' and x that have a Hamming distance greater than one (which means they have at least $N - 1$ matching elements). As such, in discrete diffusion models, neural networks trained to approximate ratios of the data distribution $\frac{Q_{T-s}(x')}{Q_{T-s}(x)}$ for neighboring x' and x can be used to implement an arbitrarily good approximation to the actual reverse process [3].

3. The Diffusion Loss

As discussed in the main text, a diffusion model is trained by minimizing the distributional distance between the joint distributions of the forward process $Q_{0,\ldots,T}$ and our learned approximation to the reverse process $P^{\theta}_{0,\ldots,T}$,

$$\mathcal{L}_{DN}(\theta) = D\left(Q_{0,\ldots,T}(\cdot)||P^{\theta}_{0,\ldots,T}(\cdot)\right) \tag{B36}$$

the Markovian nature of Q can be taken advantage of to simplify Eq. (B36) into a layerwise form,

$$\mathcal{L}_{DN}(\theta) + C = -\sum_{t=1}^{T} \mathbb{E}_{Q(x_{t-1},x_t)}\left[\log\left(P_\theta(x_{t-1}|x_t)\right]\right. \tag{B37}$$

where C does not depend on θ. For denoising algorithms that operate in the infinitesimal limit, the simple form of P_θ allows for $\mathcal{L}_{DN}$ and its gradients to be computed exactly.

a. *A Monte-Carlo gradient estimator*

In the case where $P_\theta\left(x^{t-1}|x^t\right)$ is an EBM, there exists no simple closed-form expression for $\nabla_\theta \mathcal{L}_{DN}(\theta)$. In that case, one must employ a Monte Carlo estimator to approximate the gradient. This estimator can be derived directly by taking the gradient of Eq. (B37),

$$\nabla_\theta \mathcal{L}_{DN}(\theta) = -\sum_{t=1}^{T} \mathbb{E}_{Q(x^{t-1},x^t)}\left[\nabla_\theta \log\left(P_\theta(x^{t-1}|x^t)\right)\right] \tag{B38}$$

If we have an EBM parameterization for $P_\theta\left(x^{t-1}|x^t\right)$ this may be simplified further. Specifically, given the latent variable from Eq. 8 in the main text, the gradient of log-likelihood may be simplified to,

$$\nabla_\theta \log\left(P_\theta(x^{t-1}|x^t)\right) = \mathbb{E}_{P_\theta(x^{t-1},z^{t-1}|x^t)}\left[\nabla_\theta \mathcal{E}^m_{t-1}\right] - \mathbb{E}_{P_\theta(z^{t-1}|x^{t-1},x^t)}\left[\nabla_\theta \mathcal{E}^m_{t-1}\right] \tag{B39}$$

Inserting this into Eq. (B38) yields the final result given in Eq. 14 in the article.

4. Simplification of the Energy Landscape

As the forward process timestep is made smaller, the energy landscape of the EBM-based approximation to the reverse process becomes simpler. A simple 1D example serves as a good demonstration of this concept. Consider the marginal energy function,

$$\mathcal{E}^{\theta}_{t-1}\left(x_{t-1}\right) = \left(x^2_{t-1} - 1\right)^2 \tag{B40}$$

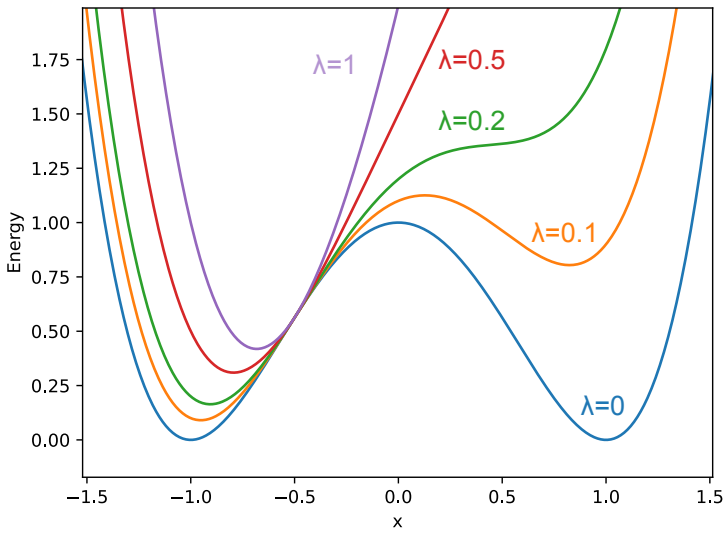

FIG. 7. **Conditioning of the energy landscape** As λ is increased, the energy landscape is reshaped from a strongly bimodal distribution towards a simple Gaussian centered at $x_t = -0.5$. The latter is much easier to sample from.

and a forward process energy function that corresponds to Gaussian diffusion (Eq. (B1)),

$$\mathcal{E}^f_{t-1}(x_{t-1}, x_t) = \lambda \left(\frac{x_{t-1}}{x_t} - 1 \right)^2 \tag{B41}$$

The parameter λ scales inversely with the size of the forward process timestep; that is, $\lim_{\Delta t \to 0} \lambda = \infty$.

The reverse process conditional energy landscape is then $\mathcal{E}^\theta_{t-1} + \mathcal{E}^f_{t-1}$. The effect of λ on this is shown in Fig. 7.

The energy landscape is bimodal at $\lambda = 0$ and gradually becomes distorted towards an unimodal distribution centered at x_t as λ increases. This reshaping is intuitive, as shortening the forward process timestep should more strongly constrain x_{t-1} to x_t.

5. Conditional Generation

The denoising framework can be adapted for conditional generation tasks, such as generating MNIST digits given a specific class label. In principle, this is very simple: we concatenate the target (in our case, the images) and a one-hot encoding of the labels into a contiguous binary vector and treat that whole thing as our training data on which we train the denoising model as described above.

In this case, the visible nodes of the Boltzmann machine are partitioned into "pixel nodes" V_X and "label nodes" V_L. All visible nodes come in pairs of input and output nodes (drawn in blue and green resp. in Fig. 3 in the main paper body and Fig. 9 below), so the set of visible nodes now consists of $V_X^{\text{in}}, V_X^{\text{out}}, V_L^{\text{in}}$, and V_L^{out}.

The training procedure works the same way as before, just using this label-augmented data. We obtain the noised training images X_n and labels L_n by noising each entry of X_0 and L_0 resp. independently using the forward process described in subsection B 1 b. Then we train the nth step model $P_{\theta_n}\big(V_X^{\text{out}} = x, V_L^{\text{out}} = l \,|\, V_X^{\text{in}} = x', V_L^{\text{in}} = l'\big)$ to approximate (in terms Kullback-Leibler divergence) the distribution $\mathbb{P}\big(X_n = x, L_n = l \,|\, X_{n+1} = x', L_{n+1} = l'\big)$.

At inference time, we have two cases:

- Unconditional inference proceeds as with regular denoising. We pass the pixel and label values backward through all the step models, and at the end, we record the pixel values.

- For conditional generation we clamp all label output nodes V_L^{out} in all step models to l_0 and sample $\widehat{X}_n \sim P_{\theta_n}\big(V_X^{\text{out}} = \cdot \,|\, V_X^{\text{in}} = \widehat{X}_{n+1}, V_L^{\text{out}} = V_L^{\text{in}} = l_0\big)$, where l_0 is an unnoised label and $\widehat{X}_{n+1}$ is the output of $P_{\theta_{n+1}}$ (or uniform noise if $n = N$).

Note that all step models except θ_0 will be trained on somewhat noised labels, so they might never have seen a pristine unnoised label during training (if there are 10 classes and five label repetitions, a strongly noised label has an approximately 10×2^{-50} chance of being a valid unnoised label). However, during conditional inference, the models will have their label nodes clamped to an unnoised label l_0, and they may not know how this should influence the generated image (and this problem would only be exacerbated if we clamped to a noised label instead).

This issue can be mitigated by using a rate γ_X when noising image entries in the training data and a different rate γ_L for noising label entries. Recall that the higher the γ, the noisier the data will become as n increases.

We consider two extremes:

- If $\gamma_L \geq \gamma_X$, then we have the exact same problem as before.
- If $\gamma_L = 0$, then the labels in the training data are a zero-temperature distribution. This low temperature can lead to freezing, potentially negating the benefits denoising could otherwise bring.

Experimentally, we observed that settings in the ranges $\gamma_L \in [0.1, 0.3]$ and $\gamma_X \in [0.7, 1.5]$ (for models with four to 12 steps) yielded good conditional generation performance while avoiding the freezing problem.

6. Learning the marginal

If a DTM is trained to match the conditional distribution of the reverse process perfectly, the learned energy function $\mathcal{E}^\theta_{t-1}$ is the energy function of the true marginal distribution, that is, $\mathcal{E}^\theta_{t-1}(x) \propto \log Q(x^{t-1})$. To show this, we start by applying the Bayes' rule to the learned reverse process conditional in the limit that it perfectly matches the true reverse process,

$$\frac{Q(x^t|x^{t-1})Q(x^{t-1})}{Q(x^t)} = \frac{1}{Z(\theta, x^t)} e^{-\left(\mathcal{E}^f_{t-1}\left(x^{t-1}, x^t\right) + \mathcal{E}^\theta_{t-1}\left(x^{t-1}, \theta\right)\right)} \tag{B42}$$

defining the distribution,

$$H(x^{t-1}) = \frac{1}{Z(\theta)} \sum_{z^{t-1}} e^{-\mathcal{E}^\theta_{t-1}\left(x^{t-1}, z^{t-1}, \theta\right)} \tag{B43}$$

$$Z(\theta) = \sum_{x^{t-1}, z^{t-1}} e^{-\mathcal{E}^\theta_{t-1}\left(x^{t-1}, z^{t-1}, \theta\right)} \tag{B44}$$

extracting the forward process from the RHS of Eq. (B42) and using Eq. (B43),

$$\frac{Q(x^{t-1})}{Q(x^t)} = \frac{Z(\theta)Z}{Z(\theta, x^t)} H(x^{t-1}) \tag{B45}$$

Eq. (B45) can easily be re-arranged into a form where the LHS depends only on x^t, and the RHS depends only on x^{t-1}. From this, we can deduce,

$$\frac{Q(x^{t-1})}{H(x^{t-1})} = c \tag{B46}$$

from the fact that Q and H are both normalized, we can find that $c = 1$, which establishes the desired equivalence.

Appendix C: Hardware accelerators for EBMs

In this work, we focus on a hardware architecture for EBMs that are naturally expressed as Probabilistic Graphical Models (PGMs). In a PGM-EBM, the random variables involved in the model map to the nodes of a graph, which are connected by edges that indicate dependence between variables.

PGMs form a natural basis for a hardware architecture because they can be sampled using a modular procedure that respects the graph's structure. Specifically, the state of a PGM can be updated by iteratively stepping through each node of the graph and resampling one variable at a time, using only information about the current node and

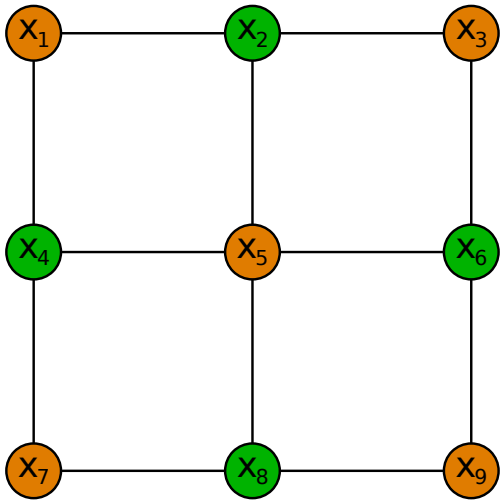

FIG. 8. **Chromatic Gibbs Sampling** A schematic view of an abstract hardware accelerator for a simple EBM. Each of the model's variables is assigned to a node. Each node is capable of receiving information from its neighbors and updating its state according to the appropriate conditional distribution. Since each node's update distribution only depends on the state of its neighbors and because nodes of the same color do not neighbor each other, they can all be updated in parallel.

its immediate neighbors. Therefore, if a PGM is local, sparse, and somewhat heterogeneous, a piece of hardware can be built to efficiently sample from it that involves spatially arraying probabilistic sampling circuits that interact with each other cheaply via short wires.

This local PGM sampler represents a type of compute-in-memory approach, where the state of the sampling program is spatially distributed throughout the array of sampling circuitry. Since the sampling circuits only communicate locally, this type of computer will spend significantly less energy on communication than one built on a Von-Neumann-like architecture, which constantly shuttles data between compute and memory.

Formally, the algorithm that defines this modular sampling procedure for PGMs is called Gibbs sampling. In Gibbs sampling, samples are drawn from the joint distribution $p(x_1, x_2, \ldots, x_N)$ by iteratively updating the state of each node conditioned on the current state of its neighbors. For the i^{th} node, this means sampling from the distribution,

$$x_i[t+1] \sim p(x_i|nb(x_i)[t]). \tag{C1}$$

This procedure defines a Markov chain whose stationary distribution can be easily controlled by adjusting the conditional update distributions of each node (see the next section for an example). Starting from some random initialization, this iterative update must be applied potentially many times to all graph nodes before the Markov chain converges to the desired stationary distribution, allowing us to draw samples from it.

Gibbs sampling allows for any two nodes that are not neighbors to be updated in parallel, meaning that the state can be updated in batches corresponding to different color groups of the graph. For a more thorough explanation of how Gibbs sampling works, see [4].

Fig. 8 shows a simple example of a PGM with two color groups that would be amenable to Gibbs sampling. Since x_1 is only connected to x_2 and x_4, the update rule from Eq. (C1) would take the form,

$$x_1[t+1] \sim p(x_1|x_4[t], x_2[t]) \tag{C2}$$

If the joint distribution had sufficient structure such that the conditional for each node had the same form, a piece of hardware could be built to sample from this PGM by building a 3x3 grid of sampling circuits that communicate only with their immediate neighbors.

1. Quadratic EBMs

The primary constraint around building a hardware device that implements Gibbs sampling is that the conditional update given in Eq. (C1) must be efficiently implementable. Generally, this means that one wants it to take a form that is "natural" to the hardware substrate being used to build the computer.

To satisfy this constraint, it is generally necessary to limit the types of joint distributions that a hardware device can sample from. An example of such a restricted family of distributions is quadratic EBMs.

Quadratic EBMs have energy functions that are quadratic in the model's variables, which generally leads to conditional updates computed by biasing a simple sampling circuit (Bernoulli, categorical, Gaussian, etc.) with the output of a linear function of the neighbor states and the model parameters. These simple interactions are efficient to implement in various types of hardware. As such, Quadratic EBMs have been the focus of most work on hardware accelerators for Gibbs sampling to date.

In the main text, we discuss Boltzmann machines, which involve only binary random variables and are the most basic form of quadratic EBM. The Conditional Update for Boltzmann Machines requires biasing a Bernoulli random variable according to a sigmoid function of a linear combination of the model parameters and the binary neighbor states, as shown in the main text, Eq. 11. This conditional update is efficiently implementable using an RNG with a sigmoidal bias and resistors, as discussed in section K.

Here, we will touch on a few other types of quadratic EBM that are more general. Although the experiments in this paper focused on Boltzmann machines, they could be trivially extended to these more expressive classes of distributions.

a. *Potts models*

Potts models generalize the concept of Boltzmann machines to k-state variables. They have the energy function,

$$E(x) = \sum_{i,j=1}^{N} \sum_{m,n=1}^{M} x_m^i J_{mn}^{ij} x_n^j + \sum_{i=1}^{N} \sum_{m=1}^{M} h_m^i x_m^i \tag{C3}$$

$$J_{mn}^{ii} = 0 \tag{C4}$$

x_m^i is a one-hot encoding of the state of variable x^i,

$$x_m^i \in \{0, 1\} \tag{C5}$$

$$\sum_m x_m^i = 1 \tag{C6}$$

which implies that $x_m^i = 1$ for a single value of m, and is zero otherwise. The distribution of any individual variable conditioned on it's Markov blanket is,

$$p(x_m^i = 1 | \text{mb}(x^i)) = \frac{1}{Z} \exp\left(-\beta \left(\sum_{j \in \text{mb}(x^i), n} J_{mn}^{ij} x_n^j + \sum_{j \in \text{mb}(x^i), n} x_n^j J_{nm}^{ji} + h_m^i \right)\right) \tag{C7}$$

In the case that J has the symmetry,

$$J_{mn}^{ij} = J_{nm}^{ji} \tag{C8}$$

this reduces to,

$$p(x_m^i = 1 | \text{mb}(x^i)) \propto \frac{1}{Z} e^{-\theta_m^i} \tag{C9}$$

$$\theta_m^i = \beta \left(2 \sum_{j \in \mathrm{mb}(x^i), n} J_{mn}^{ij} x_n^j + h_m^i \right) \tag{C10}$$

The parameters θ are defined to make it clear that this is a *softmax* distribution.

Therefore, to build a hardware device that samples from Potts models using Gibbs sampling, one would have to build a softmax sampling circuit parameterized by a linear function of the model weights and neighbor states. Potts model sampling is slightly more complicated than Boltzmann machine sampling, but it is likely possible.

b. *Gaussian-Bernoulli EBMs*

Gaussian-Bernoulli EBMs extend Boltzmann machines to continuous, binary mixtures. In general, this type of model can have continuous-continuous, binary-binary, and binary-continuous interactions. For simplicity, if we consider only binary-continuous interactions, the energy function may be written as,

$$E(v,h) = \sum_{i=1}^{N_v} \frac{(v_i - b_i)^2}{2\sigma_i^2} - \sum_{i=1}^{N_v} \sum_{j=1}^{N_h} \frac{v_i W_{ij} h_j}{\sigma_i^2} - \sum_{j=1}^{N_h} c_j h_j, \tag{C11}$$

where $v_i \in \mathbb{R}$ are continuous variables with biases b_i and variances σ_i^2, $h_j \in \{-1, 1\}$ are binary variables with biases c_j, and W_{ij} are interaction weights.

Due to the structure of the energy function, the update rule for the continuous variables corresponds to drawing a sample from a Gaussian distribution with a mean that is a linear function of the neighbor states,

$$p(v_i \,|\, \mathrm{mb}(v_i)) = \mathcal{N}\big(\mu_i, \sigma_i^2/\beta\big), \qquad \mu_i = b_i + \sigma_i^2 \sum_{j \in \mathrm{mb}(v_i)} W_{ij} h_j. \tag{C12}$$

The binary update rule is similar to the rule for Boltzmann machines,

$$p(h_j = 1 \,|\, \mathrm{mb}(h_j)) = \sigma \left(2\beta \left(\sum_{i \in \mathrm{mb}(h_j)} \frac{v_i W_{ij}}{\sigma_i^2} + c_j \right) \right) \tag{C13}$$

Hardware implementations of Gaussian-Bernoulli EBMs are more difficult than the strictly discrete models because the signals being passed during conditional sampling of the binary variables are continuous. To pass these continuous values, they must either be embedded into several discrete variables or an analog signaling system must be used. Both of these solutions would incur significant overhead compared to the purely discrete models.

Appendix D: A hardware architecture for denoising

The denoising models used in this work exclusively modeled distributions of binary variables. The reverse process energy function (Eq. 7 in the main text) was implemented using a Boltzmann machine. The forward process energy function $\mathcal{E}_{t-1}^f$ was implemented using a simple set of pairwise couplings between x^t (blue nodes) and x^{t-1} (green nodes). The marginal energy function $\mathcal{E}_{t-1}^\theta$ was implemented using a latent variable model (latent nodes are drawn in orange) with a sparse, local coupling structure.

1. Implementation of the forward process energy function

From the exponential form of the discrete-variable forward process transition kernel given in Eq. (B20), it is straightforward to derive a Boltzmann machine-style energy function that implements the forward process,

$$\mathcal{E}_{t-1}^f = \sum_i \frac{\Gamma_i(t)}{2} x_i^t x_i^{t-1} \tag{D1}$$

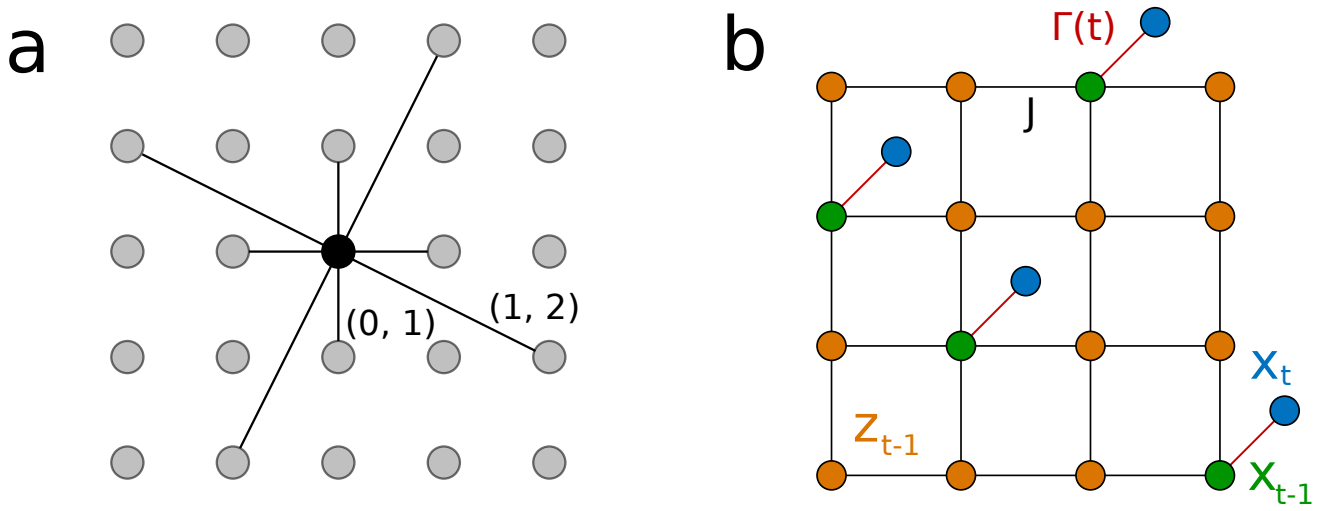

FIG. 9. **Our hardware denoising architecture (a)** An example of a possible connectivity pattern as specified in Table. II. For clarity, the pattern is illustrated as applied to a single cell; however, in reality, the pattern is repeated for every cell in the grid. **(b)** A graph for hardware denoising. The grid is subdivided at random into visible (green) nodes, representing the variables x^{t-1}, and latent (orange) nodes, representing z^{t-1}. Each visible node x_j^{t-1} is coupled to a (blue) node carrying the value from the previous step of denoising x_j^t (note that these blue nodes stay fixed throughout the Gibbs sampling).

Pattern	Connectivity
G_8	$(0,1),\ (4,1)$
G_{12}	$(0,1),\ (4,1),\ (9,10)$
G_{16}	$(0,1),\ (4,1),\ (8,7),\ (14,9)$
G_{20}	$(0,1),\ (4,1),\ (3,6),\ (8,7),\ (14,9)$
G_{24}	$(0,1),\ (1,2),\ (4,1),\ (3,6),\ (8,7),\ (14,9)$

TABLE II. Edges (ordered pairs) associated with graphs of various degrees.

where $x_t[i] \in \{-1, 1\}$ indicates the i^{th} element of the vector of random variables x_t as usual.

2. Implementation of the marginal energy function

We use a Boltzmann machine based on a grid graph to implement the marginal energy function. Our grids have both nearest-neighbor and long-range skip connections. A simple example of this is shown in Fig. 9 (a). This connectivity pattern is tiled such that every node in the bulk of the grid has the same connectivity to its neighbors. At the boundaries, connections that extend beyond the grid's edges are not formed.

Within the grid, we randomly choose some subset of the nodes to represent the data variables x_{t-1}. The remaining nodes then implement the latent variable z_{t-1}. The grid is, therefore, a deep Boltzmann machine with a sparse connectivity structure and multiple hidden layers.

We use a particular set of connectivity patterns in the experiments in this article, which are specified in Table. II. We say that node (x, y) has a connection rule of the form (a, b) if it is connected to nodes at positions $(x + a, y + b), (x - b, y + a), (x - a, y - b), (x + b, y - a)$, so each connection rule adds up to 4 edges from this node.

As explicitly stated in Eq. 7 of the article, our variational approximation to the reverse process conditional has an energy function that is the sum of the forward process energy function and the marginal energy function. Physically, this corresponds to adding nodes to our grid that implement x_t, which are connected pairwise to the data nodes implementing x_{t-1} via the coupling defined in Eq. (D1). This connectivity is shown in Fig. 9 (b).

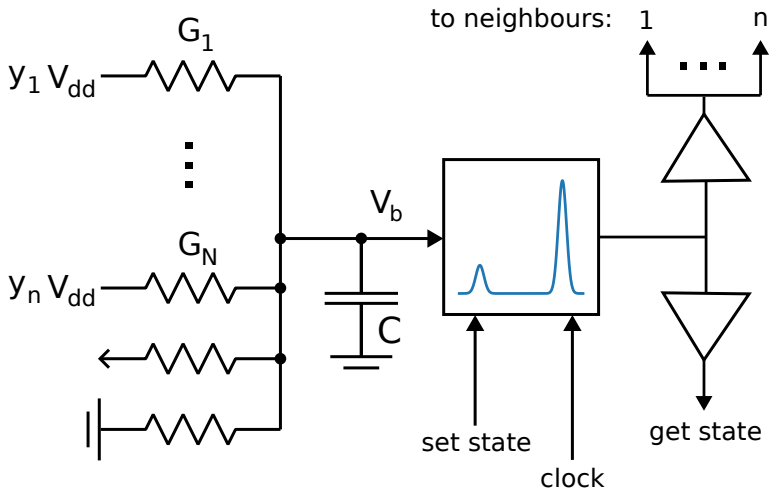

FIG. 10. **A schematic of a possible Boltzmann machine sampling cell** A linear resistor network computes a biasing voltage given the sign-corrected neighbor states $y_n = x_n \oplus s_n$. The output of this circuit biases an RNG that responds in a sigmoidal manner. This RNG processes freely when the clock is low and latches to a state when the clock is high. Upon the clock going high, the sampled state is broadcasted to the neighbors of the cell over wires.

Appendix E: Energetic analysis of the hardware architecture

Our RNG design uses only transistors and can integrate tightly with other traditional circuit components on a chip to implement a large-scale sampling system. Since there are no exotic components involved that introduce unknown integration barriers, it is straightforward to build a simple physical model to predict how this device utilizes energy.

The performance of the device can be understood by analyzing the unit sampling cell that lives on each node of the PGM implemented by the hardware. The function of this cell is to implement the Boltzmann machine conditional update, as given in Eq. 11 in the main text.

There are many possible designs for the sampling cell. The design considered here utilizes a linear analog circuit to combine the neighboring states and model weights, producing a control voltage for an RNG. This RNG then produces a random bit that is biased by a sigmoidal function of the control voltage. This updated state is then broadcast back to the neighbors. The cell must also support initialization and readout (get/set state operations). A schematic of a unit cell is shown in Fig. 8.

We provide experimental measurements of our novel RNG circuitry in the main text, which establish that random bits can be produced at a rate of $\tau_{rng}^{-1} \approx 10\,\text{MHz}$ using $\sim 350\text{aJ}$ of energy per bit. Fig. 15 (a) shows an output voltage waveform from the RNG circuit. It wanders randomly between high and low states. Critically, the bias of the RNG circuit (the probability of finding it in the high or low state) is a sigmoidal function of its control voltage, which allows for a straightforward implementation of the conditional update using linear circuitry.

The size of the RNG circuit can be used to anchor the dimensions of a future large-scale Gibbs sampling device. As shown in Fig. 15 (b), the RNG itself involves around 10 transistors and takes up $\sim 3\mu m \times 3\mu m$ on the die. It is reasonable to imagine that the whole sampling cell could fit in $4\times$ this area and have a side length of $6\mu m$. Given this area, a 1000×1000 grid of sampling cells would fit within a 6mm $\times$ 6mm chip.

Building on the measured characteristics of our RNG, we will now develop simple physical models for the remaining components of the sampling system. These models can then be combined to estimate the energy consumption of the diffusion models developed in this article running on our hardware.

1. Biasing circuit

The multiply-accumulation of the model weights and neighbor states can be performed using a resistor network, as shown in Fig. 10. The dynamics of this resistor network are described by the differential equation,

$$\sum_{j=1}^{n+2} G_j \left(V_{dd}\, y_j - V_b\right) = C\frac{dV_b}{dt} \tag{E1}$$

where $y_i = x_i \oplus s_i$ is the XOR of the neighbor state x_i with a sign bit s_i. There are n variable neighbor states and two fixed inputs ($y_{n+1} = 1$, $y_{n+2} = 0$), which are important for implementing the fixed bias term in the conditional update. V_{dd} is the supply voltage and V_b is the output voltage that biases the RNG. G_i represents the conductance of the resistor corresponding to the i^{th} input. The capacitance C represents the parasitic capacitance to ground associated with any real implementation of this circuit and is critical to forming realistic estimates of speed and energy consumption. Realistic values for an implementation of this circuit in our transistor process are shown in Fig. 11 (a).

Since this equation is first order, the dynamics exponentially relax to some fixed point V_b^∞,

$$V_b(t) = c\, e^{-t/\tau_{bias}} + V_b^\infty \tag{E2}$$

the time constant τ_{bias} is,

$$\tau_{bias} = \frac{C}{G_\Sigma} \tag{E3}$$

and the fixed point is,

$$V_b^\infty = \sum_{j=1}^{n+2} \frac{G_j}{G_\Sigma} V_{dd} y_j \tag{E4}$$

where the total conductance G_Σ is,

$$G_\Sigma = \sum_{j=1}^{n+2} G_j \tag{E5}$$

The RNG has a bias curve which takes the form,

$$\mathbb{P}\left(x_i = 1\right) = \sigma\left(\frac{V_b}{V_s} - \phi\right) \tag{E6}$$

inserting Eq. (E4) and expanding the term inside the sigmoid,

$$\frac{V_b}{V_s} - \phi = \sum_{j=1}^{n} \frac{G_j}{G_\Sigma}\frac{V_{dd}}{V_s}\left(x_j \oplus s_j\right) + \left[\frac{G_{n+1}}{G_\Sigma}\frac{V_{dd}}{V_s} - \phi\right] \tag{E7}$$

by comparison to the Boltzmann machine conditional, we can see that the first term implements the model weights (which can be positive or negative given an appropriate setting of the sign bit s_j), and the second term implements a bias.

The static power drawn by this circuit can be written in the form,

$$P^\infty = \frac{C}{\tau_{bias}} V_{dd}^2 (1-\gamma)\gamma \tag{E8}$$

where $0 \leq \gamma \leq 1$ is the input-dependent constant,

$$\gamma = \sum_{j=1}^{n+2} \frac{G_j}{G_\Sigma} y_j \tag{E9}$$

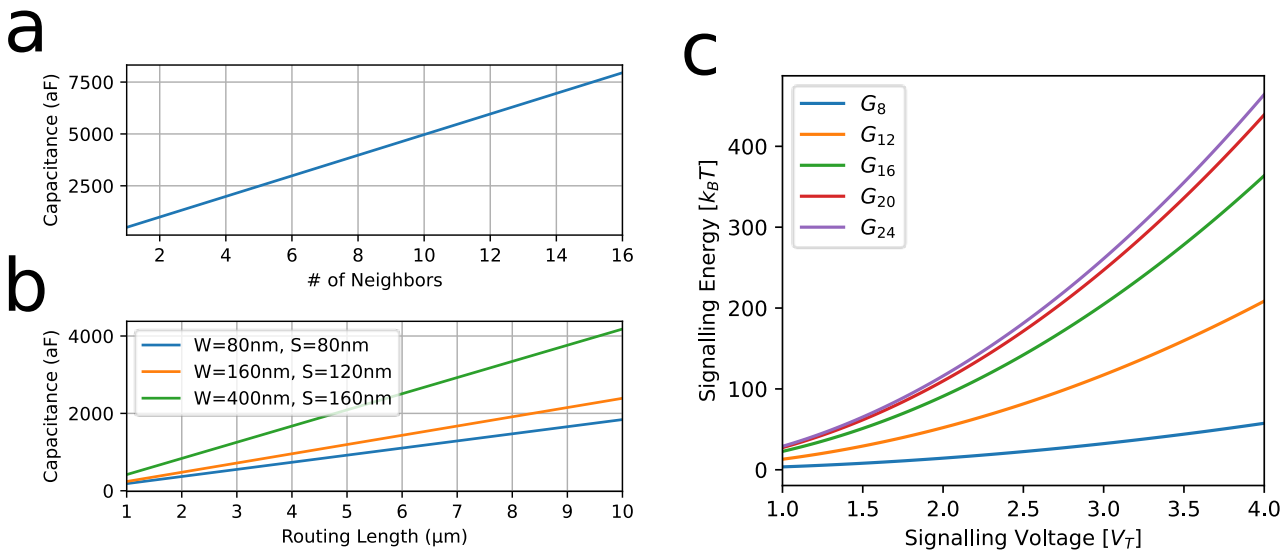

FIG. 11. **Parameters for the energy model (a)** The parasitic capacitance associated with the output node of the biasing circuit for various numbers of neighbors. These capacitances were estimated using the PDK and a layout for a real transistor implementation of the biasing circuit. **(b)** The capacitance associated with routing wires of various lengths and geometry in our process, extracted using the PDK. **(c)** The energy required for a cell to signal to all of its neighbors as a function of signaling voltage for various connectivity patterns. This energy was calculated using the routing capacitance data from (b).

This fixed point must be held while the noise generator relaxes, which means that the energetic cost of the biasing circuit is approximately,

$$\begin{aligned} E_{bias} \approx & P^{\infty}\tau_{rng} \\ = & C\frac{\tau_{rng}}{\tau_{bias}}V_{dd}^2(1-\gamma)\gamma \end{aligned} \tag{E10}$$

This is maximized for $\gamma = \frac{1}{2}$.

To avoid slowing down the sampling machine, $\frac{\tau_{rng}}{\tau_{bias}} \gg 1$. As such, ignoring the energy spent charging the capacitor $\sim \frac{1}{2}CV_b^2$ will not significantly affect the results, and the approximation made in Eq. (E10) should be accurate. The energy consumed by the bias circuit is primarily due to static power dissipation.

2. Local communication

Another significant source of energy consumption is the communication of state information between neighboring cells. In most electronic devices, signals are communicated by charging and discharging wires. Charging a wire requires the energy input,

$$E_{\text{charge}} = \frac{1}{2}C_{\text{wire}}V_{\text{sig}}^2 \tag{E11}$$

where C_{wire} is the capacitance associated with the wire, which grows with its length, and V_{sig} is the signaling voltage level.

Given the connectivity patterns shown in table II, it is possible to estimate the total capacitance C_n associated with the wire connecting a node to all of its neighbors,

$$C_n = 4\eta\ell\sum_i \sqrt{a_i^2 + b_i^2} \tag{E12}$$

where $\ell \approx 6\mu m$ is the sampling cell side length, and $\eta \approx 350\text{aF}/\mu m$ is the wire capacitance per unit length in our process, see Fig. 11 (b). a_i and b_i are the x and y components of the i^{th} connection rule, as described in section D.2.

The charging energy Eq. (E11) is plotted as a function of signaling voltage for various connectivity patterns in Fig. 11 (b).

3. Global communication

Several systems on the chip require signals to be transmitted from some central location out to the individual sampling cells. This communication involves sending signals over long wires with a large capacitance, which is energetically expensive. Here, the cost of this global communication will be taken into consideration.

a. Clocking

Although it is possible in principle to implement Gibbs sampling completely asynchronously, in practice, it is more efficient to implement standard chromatic Gibbs sampling with a global clock. A global clock requires a signal to be distributed from a central clock circuit to every sampling cell on the chip. This signal distribution is typically accomplished using a clock tree, a branching circuit designed to minimize timing inconsistencies between disparate circuit elements.

To simplify the analysis, we will consider a simple clock distribution scheme in which the clock is distributed by lines that run the entire length of each row in the grid. The total length of the wires used for clock distribution in this scheme is,

$$L_{clock} = N\,L \tag{E13}$$

where N is the number of rows in the grid, and L is the length of a row. Given this length, the energetic cost of a clock pulse can be calculated using Eq. (E11).

b. Initialization and readout

A sampling program begins by initializing every sampling cell to a specific state and ends by reading out the state of a subset of the cells for use off-chip. Both of these operations require bits to be sent over a long wire of length L from the chip's boundaries to a sampling cell in the bulk.

4. Analysis of a complete sampling program

Given the above analysis of the various subsystems, it is straightforward to construct a model of the energy consumption of a complete denoising model. Running each layer of the denoising model requires initialization of all N nodes, chromatic Gibbs sampling for K iterations, and finally, readout of the N_{data} data nodes,

$$E = T\left(E_{\text{samp}} + E_{\text{init}} + E_{\text{read}}\right) \tag{E14}$$

E_{samp} is the cost associated with the sampling iterations for each layer,

$$E_{\text{samp}} = KN\left(E_{\text{rng}} + E_{\text{bias}} + E_{\text{clock}} + E_{\text{nb}}\right) \tag{E15}$$

where E_{clock} and E_{nb} are the per-cell costs associated with clock distribution and neighbor communication, respectively.

E_{init} is the cost of initializing all the cells at the beginning of the program,

$$E_{\text{init}} = N\frac{1}{2}\eta L V_{sig}^2 \tag{E16}$$

and E_{read} is the cost of reading out the data cells at the end,

$$E_{\text{read}} = N_{\text{data}}\frac{1}{2}\eta L V_{sig}^2 \tag{E17}$$

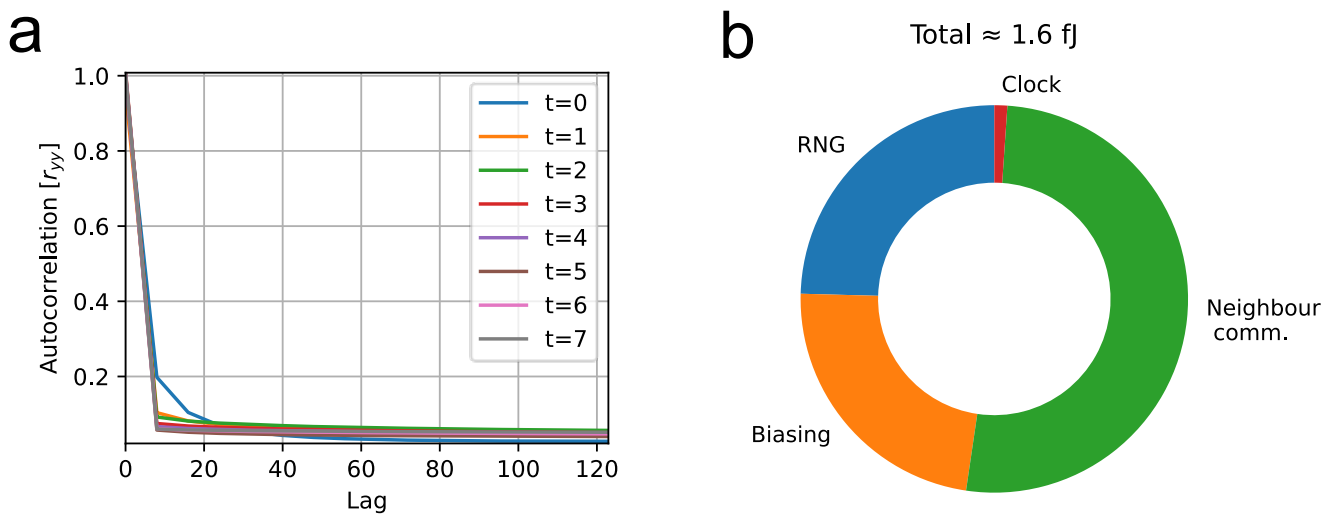

FIG. 12. (a) **Autocorrelation curve of a denoising model composed of Boltzmann machines.** Each line represents the autocorrelation of one of the Boltzmann machines that make up a fully trained denoising model.
(b) **Breakdown of the energetic cost of running a sampling cell.** Here, we take $\tau_{rng}/\tau_{bias} = 15$ and $\gamma = 1/2$. We also assume that signaling to neighbors is conducted at a voltage of $4V_T$ (where V_T is the thermal voltage k_BT/e) and the clocking and read/write operations are conducted at a signal level of $5V_T$.

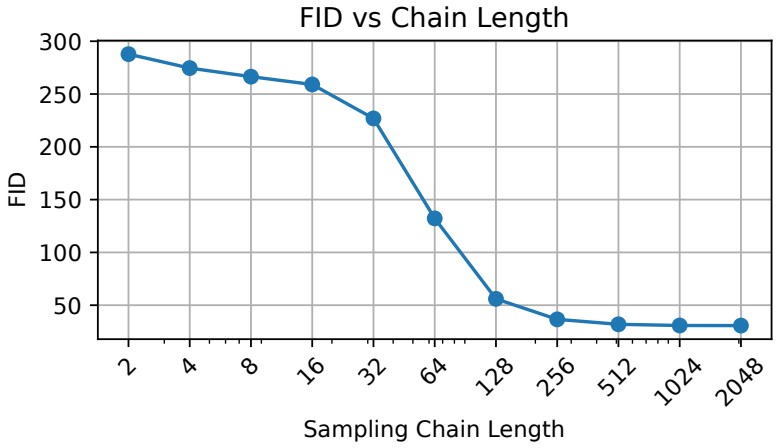

FIG. 13. The quality of output images generated by our denoising models stops improving when we sample for more than $K \approx 250$ steps.

This model was used to estimate the energy consumption of the denoising model depicted in Fig. 1 of the article. The mixing behavior for each layer in this denoising model is shown in Fig. 12 (a). All of the layers mix in tens of iterations, with the first layer decaying the most slowly. For the sake of energy calculations, we used $K = 250$ for all layers to be conservative. Fig. 13 shows that for our trained denoising models, sampling for more than $K \approx 250$ steps brings almost no additional benefit, which supports our use of this number for energy calculation. This grid used for each EBM in this model consisted of $N = 4900$ nodes that were connected using a G_{12} pattern. $N_{\text{data}} = 834$ of the nodes were assigned to data, and the rest were latent.

Given realistic choices for the rest of the free parameters of the model, the energetic cost of this denoising model is estimated to be around $1.6\,T$nJ. This is almost entirely dominated by E_{samp}, with $E_{\text{init}} + E_{\text{read}} \approx 0.01\,T$nJ. A breakdown of the various contributions to E_{samp}, along with more details about the used model parameters, is given in Fig. 12 (b).

This exact procedure was used to estimate the energy consumption of the MEBMs in Fig. 1 in the article. In this case, $T = 1$ and K were estimated from the autocorrelation data for each layer; see section L.

5. Level of realism

The model presented here captures all of the central functional units of a hardware Boltzmann machine sampler. However, the analysis was performed at a high level, and the model almost certainly underestimates the actual energy consumption of a complete device. In practice, when comparing the results of this type of calculation to a detailed analysis of a complete device design, we generally find agreement within an order of magnitude. Given that the gap between conventional methods and our novel hardware architecture is at least several orders of magnitude, this low-resolution analysis is useful, as it supports the claims made in this article without getting into every implementation detail.

Some of the discrepancies between the high-level and detailed model can be attributed to overheads associated with real circuits. A real implementation of the biasing circuit discussed in section E.1 is more complicated than the theoretical model because tunable resistors do not exist. Communications with neighboring cells over long wires require driver circuits, which consume additional energy beyond what is spent charging the line. Despite this, real circuits are bound by the same fundamental physics as the simplified models presented here. As such, the simplified models tend to estimate energy consumption within a factor of two or three of real-life values.

A real device also has additional supporting circuitry compared to our stripped-down model. In the remainder of this section, we will discuss some examples of such supporting circuitry and argue that their contributions to energy consumption at the system level ought not to be significant.

a. Programming the weights and biases

Section E.1 discusses a simple circuit that uses resistors to implement the multiply-accumulate required by the conditional update rule. Key to this is being able to tune the conductance of the resistors to implement specific sets of weights and biases (see Eq. (E7)).

Practically, implementing this tunability requires that the model parameters be stored in memory somewhere on the chip. Writing to and maintaining these memories costs energy.

Writing to the memories uses much more energy than maintaining the state. However, if writes are infrequent (program the device once and then run many sampling programs on it before writing again), then the overall cost of the memory is dominated by maintenance. Luckily, most conventional memories are specifically designed to consume as little energy as possible when not being accessed. As such, in practice, the cost of memory maintenance is small compared to the other costs associated with the sampling cells and does not significantly change the outcome shown in Fig. 12.

b. Off-chip communication

External devices have to communicate with our chip for it to be useful. The cost of this communication depends strongly on the tightness of integration between the two systems and is impossible to reason about at an abstract level. As such, the analysis of communication here (as in Section E 3 b) was limited to the cost of getting bits out to the edge of our chip, which is a lower bound on the actual cost.

However, we have found that a more detailed analysis, which includes the cost of communication between two chips mediated by a PCB, does not significantly change the results at the system level. The fundamental reason for this is that sampling programs for complex models run for many iterations before mixing and sending the results back to the outside world. This is reflected in the discrepancy between E_{samp} and $E_{\text{init}} + E_{\text{read}}$ found in section E.4.

c. Supporting circuitry

Any real chip has digital and analog supporting circuitry that provides basic functionality, such as clocking and communication, allowing the rest of the chip to function correctly. The fraction of the energy budget spent on this supporting circuitry generally depends on its size compared to the core computer. Due to the heterogeneity of our architecture, it is possible to share most of the supporting circuitry among many sampling cells, which dramatically reduces the per-cell cost. As such, the energy cost of the supporting circuitry is not significant at the system level.

Appendix F: Energy analysis of GPUs

All experiments shown in Fig. 1 in the article were conducted on NVIDIA A100 GPUs. The empirical estimates of energy were conducted by drawing a batch of samples from the model and measuring the GPU energy consumption and time via Zeus [5]. The theoretical energy estimates were derived by taking the number of model FLOPS (via JAX and PyTorch's internal estimators) and plugging them into the NVIDIA GPU specifications (19.5 TFLOPS for Float32 and 400W). The empirical measurements are compared to theoretical estimates for the VAE in Table III, and the empirical measurements show good alignment with the theoretical.

FID	Empirical Efficiency	Theoretical Efficiency
30.5	6.1×10^{-5}	2.3×10^{-5}
27.4	1.5×10^{-4}	0.4×10^{-4}
17.9	2.5×10^{-3}	1.7×10^{-3}

TABLE III. Comparing theoretical vs empirical energy consumption for a VAE on a GPU. Energy efficiencies are reported in units of joules per sample.

The models were derived from available implementations and are based on small versions of ResNet [6] and UNet [7] style architectures. Their FID performance is consistent with published literature values [8–10]. The goal is not to achieve state of the art performance, but to represent the relative scales of energy consumption of the algorithms. For a direct simulation of the Ising models on a GPU, theoretical efficiency on the order of 10^{-4} joules per sample comparable in performance and efficiency to the small VAE.

The reader may be surprised to see that the diffusion model is substantially less energy-efficient than the VAE given the relative dominance in image generation. However, two points should be kept in mind. First, while VAE remains a semi-competitive model for these smaller datasets, this quickly breaks down. On larger datasets, a FID performance gap usually exists between diffusion models and VAEs. Second, these diffusion models (based on the original DDPM [2]) have performance that can depend on the number of diffusion time steps. So, not only is the UNet model often larger than a VAE decoder, but it also must be run dozens to thousands of times in order to generate a single sample (thus resulting in multiple orders of magnitude more energy required). Modern improvements, such as distillation [11], may move the diffusion model energy efficiency closer to the VAE's.

Appendix G: Autocorrelation and mixing time

We monitor the mixing of our Gibbs samplers via the autocorrelation of a low-dimensional projection of the Markov chain state.

Let $\{x[j]\}_{j\geq 0}$ be a discrete-time Markov chain on a finite state space $\{1, \dots, d\}$, with time-homogeneous transition kernel $P = (p_{xy})_{1\leq x,y\leq d}$ given by

$$p_{xy} = \mathbb{P}(x[j+1] = y \mid x[j] = x).$$

In our setting, $x[j]$ is the state of the Boltzmann machine at Gibbs iteration j.

We fix a function (projection) $f : \{1, \dots, d\} \to \mathbb{R}$ and define the scalar observable

$$y[j] = f(x[j]).$$

We then define the mean

$$\mu \equiv \mathbb{E}\big[y[j]\big] \tag{G1}$$

and the (normalized) autocorrelation function

$$r_{yy}[k] = \frac{\mathbb{E}\big[(y[j]-\mu)\,(y[j+k]-\mu)\big]}{\mathbb{E}\big[(y[j]-\mu)^2\big]}, \qquad k \in \mathbb{N}. \tag{G2}$$

Here $\mathbb{E}[\cdot]$ denotes expectation with respect to the joint law of the Markov chain. In practice, we approximate this expectation by averaging over time and across multiple independent Gibbs sampling chains.

For all experiments in this article, we take f to be the encoder network used in the FID computation, applied to the

visible states of the Boltzmann machine. This choice is largely arbitrary: we found that much simpler embeddings, such as random linear projections $y[j] = Ax[j]$, behave similarly well for the purposes of autocorrelation-based mixing diagnostics.

Spectral decomposition and decay of autocorrelation

We now briefly recall how the decay rate of autocorrelation is tied to the spectral properties of the transition kernel P, and hence to the mixing time of the Markov chain. Proofs of the statements below can be found in standard references such as [12].

Assume the Markov chain $\{x[j]\}_{j\geq 0}$ is

- **irreducible**, i.e. any state can be reached from any other in a finite number of steps, and
- **aperiodic**, i.e. for any $x \in \{1, \dots, d\}$ there exists $T \in \mathbb{N}$ such that for all $t \geq T$, $\mathbb{P}(x[t] = x \mid x[0] = x) > 0$.

Since the chain is finite and irreducible, there exists a unique stationary distribution π satisfying

$$\pi = \pi P.$$

Furthermore, aperiodicity implies that, for any initial distribution ψ_0 of $x[0]$, the law ψ_t of $x[t]$ converges to π as $t \to \infty$:

$$\psi_t = \psi_0 P^t \;\longrightarrow\; \pi \qquad \text{as } t \to \infty.$$

Assume further that P is diagonalizable with real eigenvalues (which is always true if the chain is reversible, like in the case of Gibbs sampling), and write its eigendecomposition as

$$P = U^{-1} \Sigma\, U,$$

where Σ is diagonal, and its entries are ordered

$$1 = \sigma_1 > \sigma_2 \geq \cdots \geq \sigma_d \geq 0.$$

We denote by $U(i, x)$ the entry in the ith row and xth column of U (and similarly for U^{-1}). The first left eigenvector of P (the first row of U) is the stationary distribution,

$$\pi = \pi P = U(1, \cdot),$$

and the first right eigenvector is a column of ones,

$$U^{-1}(\cdot, 1) = \mathbf{1}.$$

Let $f : \{1, \dots, d\} \to \mathbb{R}$ be the same function as above, and write

$$\mu_0^f \equiv \mathbb{E}_{Y\sim\psi_0}\big[f(Y)\big] = \mathbb{E}\big[f(x[0])\big], \qquad \mu_\infty^f \equiv \mathbb{E}_{Y\sim\pi}\big[f(Y)\big] = \lim_{t\to\infty} \mathbb{E}\big[f(x[t])\big], \tag{G3}$$

where ψ_0 is the initial distribution of $x[0]$.

Let δ_k denote the column vector with a 1 in position k and 0 elsewhere. Using the eigendecomposition of P, the transition probabilities can be written as

$$(\delta_{x_0}^T P^t)(x) = \sum_{j=1}^{d} U^{-1}(x_0, j)\, \sigma_j^t\, U(j, x).$$

Thus,

$$\begin{aligned}
\mathbb{E}\big[f(x[0])f(x[t])\big] &= \sum_{x_0=1}^{d} f(x_0)\,\mathbb{P}\big[x[0]=x_0\big]\,\mathbb{E}\big[f(x[t]) \mid x[0]=x_0\big] \\
&= \sum_{x_0=1}^{d}\sum_{x=1}^{d} f(x_0)f(x)\,\psi_0(x_0)\,(\delta_{x_0}^T P^t)(x) \\
&= \sum_{x_0=1}^{d}\sum_{x=1}^{d} f(x_0)f(x)\,\psi_0(x_0)\sum_{j=1}^{d} U^{-1}(x_0,j)\,\sigma_j^t\,U(j,x) \\
&= \left(\sum_{x_0=1}^{d}\sum_{x=1}^{d} f(x_0)f(x)\,\psi_0(x_0)\,U^{-1}(x_0,1)\,\sigma_1^t\,U(1,x)\right) \\
&\quad + \sum_{j=2}^{d}\sigma_j^t\sum_{x_0=1}^{d}\sum_{x=1}^{d} f(x_0)f(x)\,\psi_0(x_0)\,U^{-1}(x_0,j)\,U(j,x).
\end{aligned} \tag{G4}$$

Using $U^{-1}(\cdot,1)=\mathbf{1}$ and $U(1,\cdot)=\pi$, the first term simplifies to

$$\sum_{x_0=1}^{d}\sum_{x=1}^{d} f(x_0)f(x)\,\psi_0(x_0)\,\pi(x) = \mu_0^f\mu_\infty^f.$$

We can therefore write

$$\mathbb{E}\big[f(x[0])f(x[t])\big] = \mu_0^f\mu_\infty^f + \sum_{j=2}^{d}\sigma_j^t c_j, \tag{G5}$$

where the coefficients c_j are constants (depending on f and ψ_0 but not on t). For large t, the terms with smaller eigenvalues become negligible compared to the contribution from σ_2, so the covariance

$$\mathbb{E}\big[f(x[0])f(x[t])\big] - \mu_0^f\mu_\infty^f$$

decays asymptotically like σ_2^t. In particular, under stationarity ($\psi_0=\pi$ so that $\mu_0^f=\mu_\infty^f=\mu$), the autocorrelation $r_{yy}[t]$ defined in (G2) decays approximately as

$$r_{yy}[t] \approx C\,\sigma_2^t$$

for some constant C depending on f.

From eigenvalues to mixing time

The spectral quantity σ_2 is directly related to the mixing time of the Markov chain. Recall that the mixing time is defined by

$$\tau(\varepsilon) = \min\left\{t \geq 0 : \max_{\psi_0} \|\psi_0 P^t - \pi\|_{\mathrm{TV}} \leq \varepsilon\right\},$$

where

$$\|\mu-\nu\|_{\mathrm{TV}} = \frac{1}{2}\sum_{x=1}^{d}|\mu(x)-\nu(x)|$$

denotes the total variation distance and $\varepsilon>0$ is a prescribed tolerance.

Using the eigendecomposition $P = U^{-1}\Sigma U$, we can write

$$\psi_0 P^t(x) = \sum_{j=1}^{d} \big(\psi_0 \cdot U^{-1}(\cdot, j)\big)\, \sigma_j^t\, U(j, x),$$

where ψ_0 is treated as a row vector and $\psi_0 \cdot U^{-1}(\cdot, j)$ denotes the scalar product with the jth column of U^{-1}. Using $\psi_0 \cdot U^{-1}(\cdot, 1) = 1$ and $U(1, \cdot) = \pi$, we obtain

$$\begin{aligned}
\|\psi_0 P^t - \pi\|_{\mathrm{TV}} &= \frac{1}{2}\sum_{x=1}^{d} \left| \pi(x) - \sum_{j=1}^{d} \big(\psi_0 \cdot U^{-1}(\cdot, j)\big)\, \sigma_j^t\, U(j, x) \right| \\
&= \frac{1}{2}\sum_{x=1}^{d} \left| \pi(x) - \pi(x) + \sum_{j=2}^{d} \big(\psi_0 \cdot U^{-1}(\cdot, j)\big)\, \sigma_j^t\, U(j, x) \right| \\
&\leq \frac{1}{2}\sum_{x=1}^{d}\sum_{j=2}^{d} \big|\psi_0 \cdot U^{-1}(\cdot, j)\big|\, |\sigma_j^t|\, |U(j, x)| \\
&= \sum_{j=2}^{d} \sigma_j^t\, a_j \;\leq\; \sigma_2^t \sum_{j=2}^{d} a_j,
\end{aligned}$$

where $a_j \geq 0$ are constants that depend on ψ_0 and on the eigenvectors of P. Therefore, the total variation distance decays at least as fast as σ_2^t, and we obtain the upper bound

$$\tau(\varepsilon) \;\leq\; \frac{\log(\varepsilon) - \log\Big(\sum_{j=2}^{d} a_j\Big)}{\log(\sigma_2)}.$$

Since $0 \leq \sigma_2 < 1$, the denominator $\log(\sigma_2)$ is negative, and the right-hand side is positive. The smaller the value of σ_2, the faster the Markov chain mixes, and the more rapidly the autocorrelation $r_{yy}[k]$ decays.

In summary, by empirically estimating the long-lag decay of the autocorrelation function $r_{yy}[k]$ for some informative observable $f(x)$, we effectively probe the second-largest eigenvalue σ_2 of the transition kernel and thus obtain a proxy for the mixing time of the Gibbs sampler underlying our DTM.

Appendix H: Total correlation penalty

In the main text (see Eq. 17), we explain how we utilize a total correlation penalty to encourage the latent variable EBMs employed in our model to mix rapidly. Here, we will discuss a few details of this regularizer and the method we use to control its strength adaptively.

1. Gradients of the total correlation penalty

The total correlation penalty is a convenient choice in this context because its gradients can be computed using the same samples used to estimate the gradient of the usual loss used in training, $\nabla_\theta \mathcal{L}_{DN}$. Namely, treating the factorized distribution as a constant with respect to the gradient,

$$\nabla_\theta \mathcal{L}_t^{TC} = \mathbb{E}_{Q(x^{t-1})}\left[\mathbb{E}_{d(s^{t-1}|x^t)}\left[\nabla_\theta \mathcal{E}_{t-1}^\theta\right] - \mathbb{E}_{P_\theta(s^{t-1}|x^t))}\left[\nabla_\theta \mathcal{E}_{t-1}^\theta\right]\right] \tag{H1}$$

where,

$$d(s^{t-1}|x^t) = \prod_{i=1}^{M} P_\theta(s_i^{t-1}|x^t) \tag{H2}$$

The second term in Eq. (H1) also appears in the estimator for $\nabla_\theta \mathcal{L}_{DN}$. The first term can be simplified when $\mathcal{E}_{t-1}^\theta$

has particular symmetries. For example, if $\mathcal{E}^{\theta}_{t-1}$ is a Boltzmann machine energy function (see main text Eq. 10),

$$\mathbb{E}_{d(s^{t-1}|x^t)}\left[\frac{d}{dh_i}\mathcal{E}^{\theta}_{t-1}\right] = -\beta\,\mathbb{E}_{P_\theta(s_i|x_t)}\left[s_i\right] \tag{H3}$$

$$\mathbb{E}_{d(s^{t-1}|x^t)}\left[\frac{d}{dJ_{ij}}\mathcal{E}^{\theta}_{t-1}\right] = -\beta\,\mathbb{E}_{P_\theta(s_i|x_t)}\left[s_i\right]\;\mathbb{E}_{P_\theta(s_j|x_t)}\left[s_j\right] \tag{H4}$$

Each of these terms is easy to compute given the samples used to estimate $\nabla_\theta \mathcal{L}_{DN}$.

2. Adaptive Correlation Penalty

The optimal strength of the correlation penalty λ_t may vary depending on the specific denoising step t (models for less noisy data near $t = 0$ may require stronger regularization) and may even change during training for a single-step model. Manually tuning λ_t for each of the step-models would be prohibitively expensive.

To address this, we employ an Adaptive Correlation Penalty (ACP) scheme that dynamically adjusts λ_t based on an estimate of the model's current mixing time. We use the autocorrelation of the Gibbs sampling chain, r^t_{yy}, as a proxy for mixing, as described in Section G and the main text, Eq. 18.

Our ACP algorithm monitors the autocorrelation at a lag K equal to the number of Gibbs steps used in the estimation of $\nabla_\theta \mathcal{L}_{DN}$. The goal is to adjust γ_{CP} to keep this autocorrelation below a predefined target threshold ε_{ACP}.

A simple layerwise procedure is used for this control. The inputs to the algorithm are the initial values of λ_t, a target autocorrelation threshold ε_{ACP} (e.g., 0.03), an update factor δ_{ACP} (e.g., 0.2) and a lower limit λ^{min}_t (e.g., 0.0001).

At the end of each training epoch m:

1. Estimate the current autocorrelation $a^t_m = r^t_{yy}[K]$. This estimate can be done by running a longer Gibbs chain periodically and calculating the empirical autocorrelation from the samples.
2. Set $\lambda'_t = max(\lambda^{\text{min}}_t, \lambda^{(m)}_t)$ to avoid getting stuck at 0.
3. Update λ_t for the next epoch $(m + 1)$ based on a^t_m and the previous value a^t_{m-1} (if $m > 0$):
   - If $a^t_m < \varepsilon_{\text{ACP}}$: The chain mixes sufficiently fast; reduce the penalty slightly.
   $$\lambda^{(m+1)}_t \leftarrow (1 - \delta_{\text{ACP}})\lambda'_t$$
   - Else if $a^t_m \geq \varepsilon_{\text{ACP}}$ and $a^t_m \leq a^t_{m-1}$ (or $m = 0$): Mixing is slow but not worsening (or baseline); keep the penalty strength.
   $$\lambda^{(m+1)}_t \leftarrow \lambda'_t$$
   - Else ($a^t_m > \varepsilon_{\text{ACP}}$ and $a^t_m > a^t_{m-1}$): Mixing is slow and worsening; increase the penalty.
   $$\lambda^{(m+1)}_t \leftarrow (1 + \delta_{\text{ACP}})\lambda'_t$$
4. If the proposed value $\lambda^{(m+1)}_t < \lambda^{\text{min}}_t$, then set $\lambda^{(m+1)}_t \leftarrow 0$.

Our experiments indicate that this simple feedback mechanism works effectively. While λ_t and the autocorrelation a^t_m might exhibit some damped oscillations for several epochs before stabilizing this automated procedure is vastly more efficient than performing manual hyperparameter searches for λ_t for each of the T models.

Training is relatively insensitive to the exact choice of ε_{ACP} within a reasonable range (e.g., $[0.02, 0.1]$) and δ_{ACP} (e.g., $[0.1, 0.3]$). Assuming that over the course of training the λ_t parameter settles around some value λ^*_t, one should aim for the lower bound parameter λ^{min}_t to be smaller than $\frac{1}{2}\lambda^*_t$, while making sure that the ramp-up time $\frac{\log(\lambda^*_t)-\log(\lambda^{\text{min}}_t)}{\log(1+\delta_{\text{ACP}})}$ remains small. Settings of λ^{min}_t in the range $[0.001, 0.00001]$ all produced largely the same result, the only difference being that values on the lower end of that range led to a larger amplitude in oscillations of λ_t and a^t_m, but training eventually settled for all values. An example of some ACP dynamics is shown in Fig. 14:

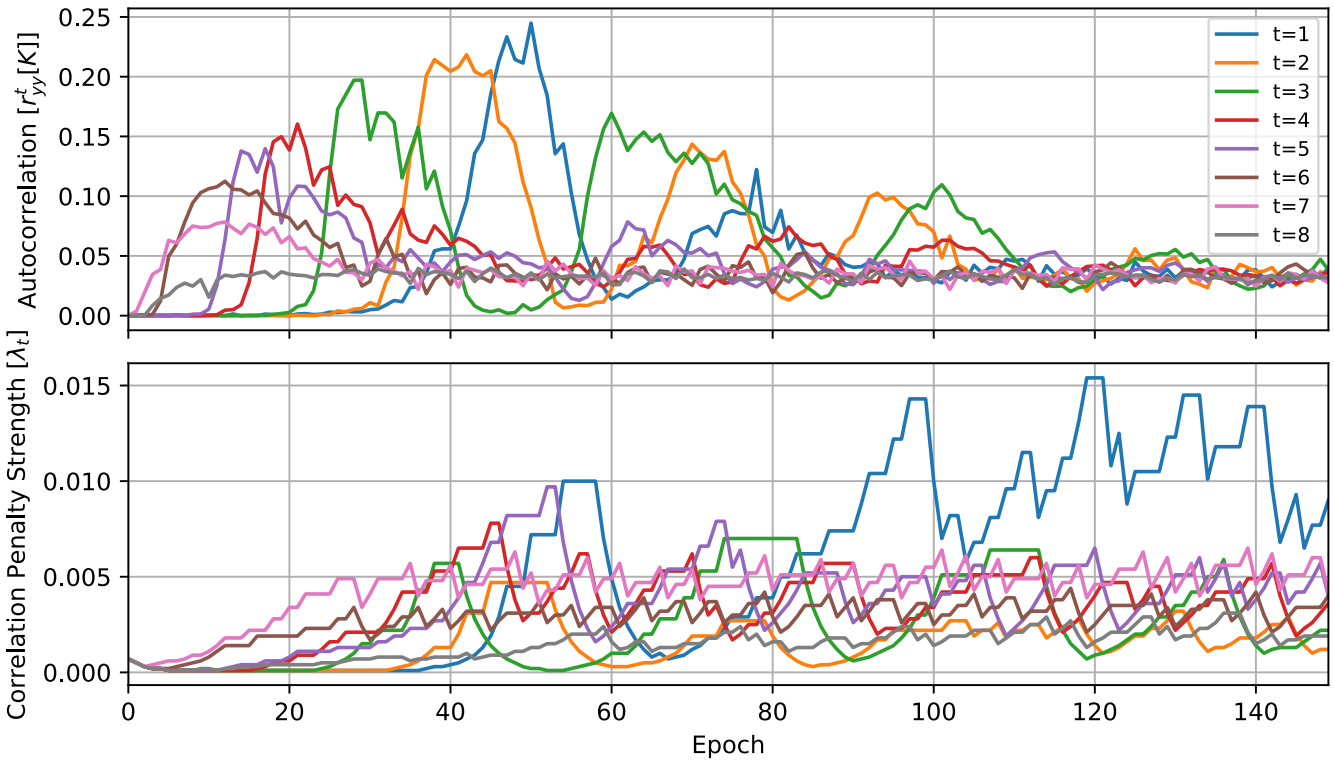

FIG. 14. **The adaptive correlation penalty** The dynamics of r^t_{yy} and λ_t over a training run. Large values of r^t_{yy} lead to increasingly large values of λ_t, which cause r^t_{yy} to decrease. The system reaches a stable configuration by the end of training.

Training on Fashion-MNIST with the typical experimental setup, we observed nearly the same performance (a FID of 28 ± 1) for all choices of ε_{ACP}, δ_{ACP} and $\lambda_t^{\min}$ in the ranges written above, so long as we trained for at least 100 epochs (with specific settings the training took longer to converge).

Appendix I: Embedding integers into Boltzmann machines

In some of our experiments, we needed to embed continuous data into binary variables. We chose to do this by representing a k-state categorical variable X_i using the sum k binary variables Z_i^k,

$$X_i = \sum_{k=1}^{K_i} Z_i^{(k)} \tag{I1}$$

where $Z_i^{(k)} \in \{0, 1\}$. These binary variables can be trivially converted into spin variables that are $\{-1, 1\}$ valued using a linear change of variables.

Energy functions that involve quadratic interactions between these categorical variables can be reduced to Boltzmann machines with local patches of all-to-all connectivity. For example, consider the energy function,

$$E(x; \theta) = -\sum_{i \neq j} w_{ij} X_i X_j - \sum_{i=1}^{d} b_i X_i \tag{I2}$$

inserting Eq. (I1), we can rewrite this in terms of quadratic interactions between the underlying spins $Z_i^{(k)}$,

$$E(z; \theta) = -\sum_{i \neq j} w_{ij} \left(\sum_{k=1}^{K_i} Z_i^{(k)} \right) \left(\sum_{l=1}^{K_j} Z_j^{(l)} \right) - \sum_i b_i \left(\sum_{k=1}^{K_i} Z_i^{(k)} \right) \tag{I3}$$

which is a standard Boltzmann machine energy function that can be run on our hardware, just like any other.

Appendix J: Deterministic embeddings for DTMs

In section V of the paper, we mention hybrid thermodynamic models, the purpose of which is to combine the flexibility of conventional neural networks (NNs) with the efficiency of probabilistic computers. For example, in the context of image generation, a small convolutional neural network can be used to map color images into a format compatible with a binary DTM. To properly take advantage of the DTM's energy efficiency, the conventional model should be at least 2 or 3 orders of magnitude smaller (e.g., in terms of parameter count or number of operations per sample) than the DTM.

There are various options for the type of conventional model one can use for the embedding, e.g., invertible models such as GLOW [13] or Normalizing Flows [14], as well as simpler solutions, such as an Autoencoder.

For our proof-of-concept for hybrid models, we used a combination of an Autoencoder and a GAN [15].

- First, we train a convolutional Autoencoder (encoder plus decoder) that maps images into a binary latent space (achieved through a combination of a sigmoid activation, a binarization penalty, and a straight-through gradient).
- Second, we train a DTM on latent embeddings of the training images. At inference time, the samples generated by the DTM are passed through the decoder to produce images.
- Thirdly, we use a GAN-like approach to fine-tune the decoder to utilize the outputs of the Boltzmann machine maximally. Specifically, the Boltzmann machine outputs are used as the noise source, which is fed into the decoder (now taking the role of the generator in the GAN architecture), and finally, a critic is trained to guide the decoder towards generating higher-quality images.

Our hybrid model achieved a FID score of ~ 60 on CIFAR10. Our DTM had 8 million parameters, the decoder had 65k, and the encoder and critic were both below 500k parameters. At inference time, only the DTM and the decoder are used. To achieve a similar performance with a conventional GAN, the decoder/generator requires about 500000 parameters.

Appendix K: Some details on our RNG

Our RNG is a digitizing comparator fed by a source of Gaussian noise. The noise source is implemented using the circuitry and principles described in [16]. The comparator is a standard design that operates in subthreshold to minimize energy consumption. The mean of the Gaussian noise is shifted before it is sent into the comparator to implement the bias control. A schematic of our RNG is shown in Fig. 15 (a).

Another example of an output voltage signal from our RNG is shown in Fig. 15 (b). The signal randomly wanders between high and low-voltage states. Suppose this signal is repeatedly observed, waiting for at least the correlation time of the circuit between observations. In that case, one will approximately draw samples from a Bernoulli distribution with a bias parameter that depends on the circuit's control voltage.

Our RNG was part of the same test chip used to carry out the experiments in [16]. The output of the RNG was fed into an amplification chain that buffered it and allowed its signal to be observed using an external oscilloscope. Fig. 15 (c) shows an image of our packaged test chip, along with a view of our RNG through a $100\times$ microscope objective.

Appendix L: MEBM experiments

Our experiments on MEBMs were conducted in the typical way [17]. We employed the same Boltzmann machine architecture as we typically use for the DTM layers, specifically $L = 70$ with G_{12} connectivity. Random nodes were chosen to represent the data, and the rest were left as latent variables (as discussed in section D).

Generating the data presented in Figs. 1 and 2 in the main text required controlling the mixing time of a trained Boltzmann machine. To achieve this, we added a fixed correlation penalty (Eq. 17 in the main text) and varied the strength to control the allowed complexity of the energy landscape.

Fig. 16 (a) shows an example of the raw autocorrelation curves produced by sampling from Boltzmann machines trained with different correlation penalty strengths. The slowest exponential decay rate (σ_2) could be estimated for

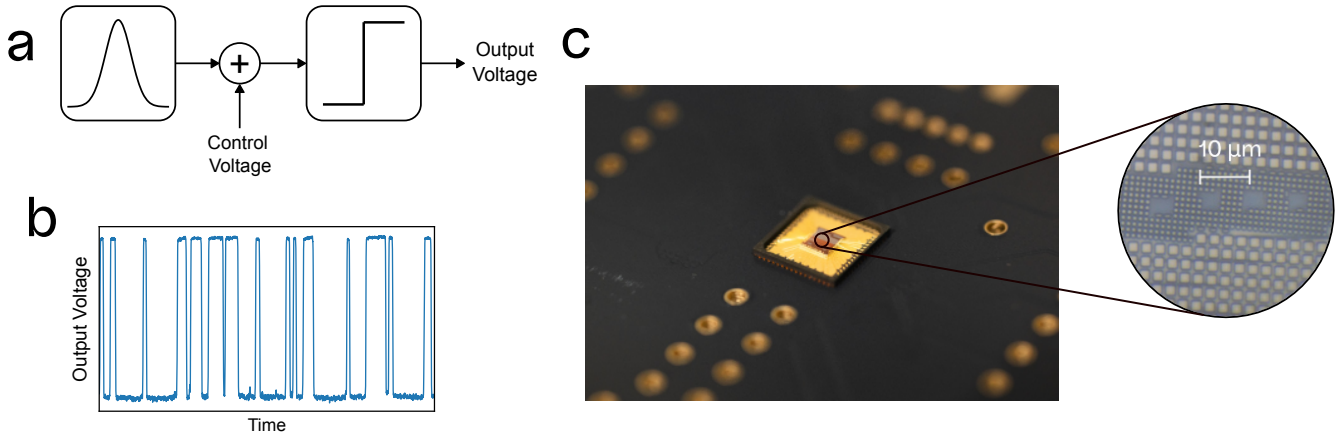

FIG. 15. **Our RNG**. **(a)** A high-level schematic of our RNG design. **(b)** Stochastic voltage signal from our RNG. The high level represents one, and the low level represents 0. The signal wanders randomly between high and low levels, with the amount of time it spends in each level controlled by the bias voltage. **(c)** An image of our packaged test chip (with the top of the package removed) assembled onto a PCB. We also show an optical microscope image of several RNG circuits on our test chip. Each circuit occupies an approximately $3 \times 3\mu m$ area on the chip.

most of the curves by fitting a line to the natural log of the autocorrelation curve at long times, see Fig. 16 (b) The two curves with the smallest correlation penalty did not reduce to simple exponential decay during the measured lag values, which means the decay rate was too long to be extracted from our data.

The exponential decay rates extracted from Fig. 16 were used as the mixing times in Fig. 2 in the article. Calling this a "mixing time" is a slight abuse of nomenclature. However, we did not think it made enough of a difference to the article's message to disambiguate (since it is an upper bound on the mixing time, as discussed in Section G).

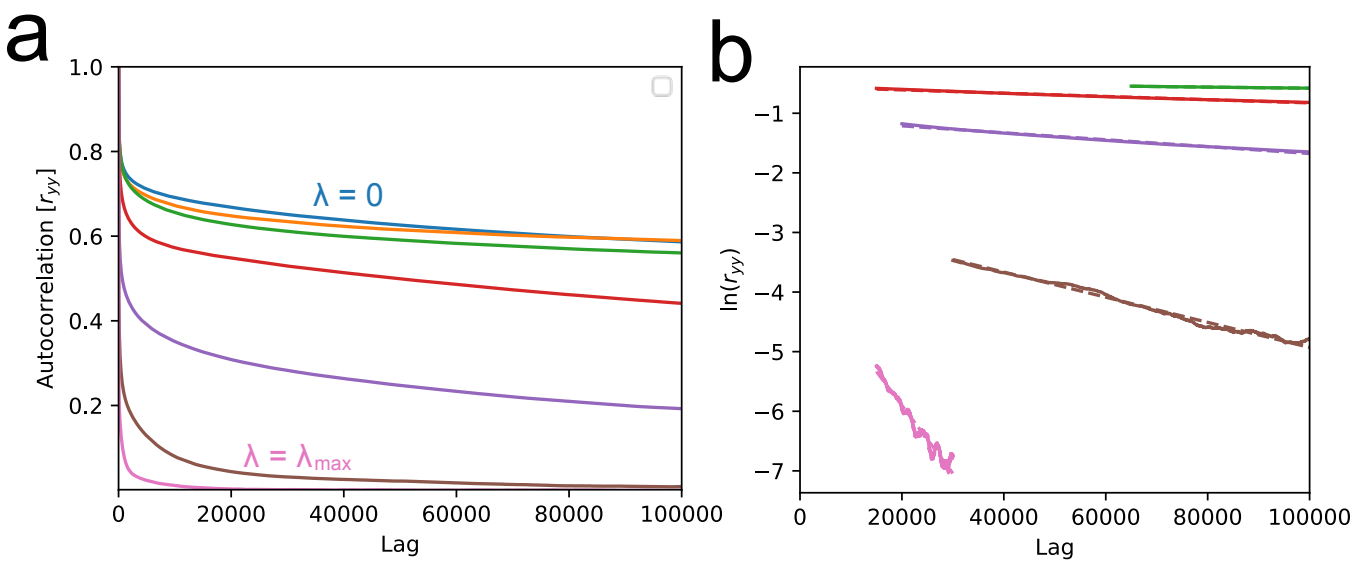

FIG. 16. **Boltzmann machine autocorrelation curves** **(a)** The raw autocorrelation data associated with Boltzmann machines trained using different values of the parameter λ. **(b)** The log of the long-time autocorrelation for some of the curves shown in (a). All curves, except for the blue and orange ones, eventually became linear.

Appendix M: Sampling time vs Performance Tradeoff

The total time it takes to sample from a DTM is $O(KT)$, where T is the number of denoising steps comprising the DTM and K. While one might use a different number of Gibbs steps at training time and inference time, it should

be noted that for DTMs trained with adaptive correlation penalty (ACP), $K_{\text{inference}}$ should be similar or only slightly higher to K_{train}. This is substantially different from the usual good practice for un-penalised EBMs. The main reason for this distinction is ACP. In particular, when training an EBM (or DTM) with ACP, the closed-loop control used in ACP will make sure that the mixing time K_{mix} of the EBM is approximately equal to K_{train}. It is always the case that using $K_{\text{inference}} >> K_{\text{mix}}$ brings no benefits compared to $K_{\text{inference}} \approx K_{\text{mix}}$ and therefore when sampling from an ACP-regularised model, using $K_{\text{inference}} \approx K_{\text{train}}$. On the other hand, for usual EBMs trained without ACP (or a similar penalty), the mixing time of the EBM can greatly exceed K_{train}. Due to the mixing-expressivity tradeoff, this at first means that the performance of this un-penalised EBM can exceed that of an ACP EBM, as long as $K_{\text{inference}} \geq K_{\text{mix}} >> K_{\text{train}}$. However, if we then keep training with $K_{\text{train}} << K_{\text{mix}}$, the performance of the EBM starts to degrade substantially, as shown in fig. 18. One may attempt to circumvent this by treating the EBM as a non-equilibrium model as explored in [18], but that introduces significant new complexity and is mathematically not well understood.

That being said, why does increasing K_{train} improve the performance of an ACP-regularised DTM or MEBM? ACP works by periodically checking whether $K_{\text{mix}} > K_{\text{train}}$. If yes, then it penalizes the model more, reducing K_{mix}, but potentially hurting the expressivity of the model. If not, then it reduces the penalty coefficient, allowing both K_{mix} and model expressivity to increase. Therefore, using a larger K_{train} lets ACP be more lenient, resulting in a more expressive model.

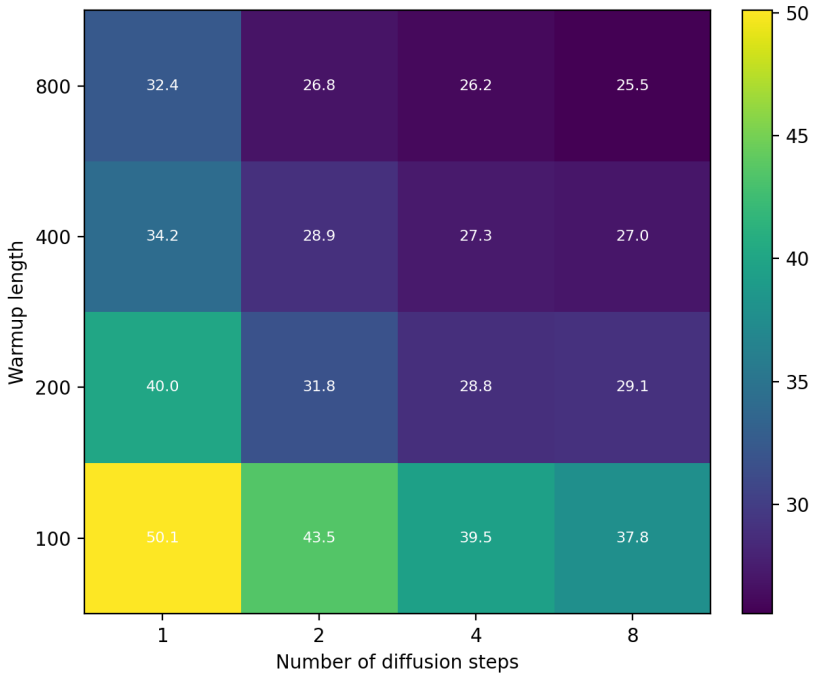

FIG. 17. The FID scores of different DTMs trained on binarised FashionMNIST. On the horizontal axis we vary the number of denoising steps T and on the vertical axis we vary the number of Gibbs steps during training K_{train}. Each diagonal (running from top-left to bottom-right) represents a constant amount of energy expenditure. For this plot, the number of Gibbs steps at inference was twice the number used during training. The reported FID values were averaged over three runs with different random seeds.

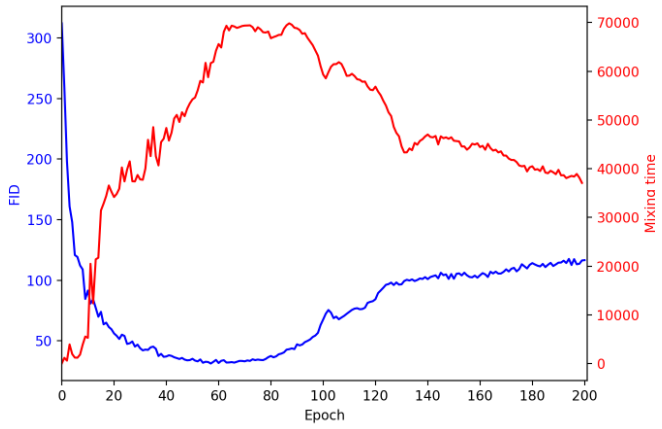

FIG. 18. FID and mixing time of an MEBM (same topology as most other experiments in this article) throughout training. As the MEBM trains and becomes more expressive (FID goes down) the mixing time increases. Around epoch 20 the model starts freezing (the mixing time becomes too long), which causes the CD gradients to become inaccurate and soon after the performance of the model starts to degrade. The mixing time subsequently also decreases somewhat, although this is likely an artefact of mode collapse rather than a sign of relaxation of the energy barriers in the model.

Appendix N: CIFAR-10 Images

FIG. 19. Images generated by the hybrid model trained on CIFAR-10 as described in section V and Appendix J. Images come in groups of 8, where each group shows the progression of the image throughout the denoising process. The denoising process operates inside a binary latent space, which is then converted into images by a small neural-network-based decoder.

[1] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, Deep unsupervised learning using nonequilibrium thermodynamics, in *International conference on machine learning* (pmlr, 2015) pp. 2256–2265.

[2] J. Ho, A. Jain, and P. Abbeel, Denoising diffusion probabilistic models, Advances in neural information processing systems **33**, 6840 (2020).

[3] A. Lou, C. Meng, and S. Ermon, Discrete diffusion modeling by estimating the ratios of the data distribution (2024), arXiv:2310.16834 [stat.ML].

[4] K. P. Murphy, *Probabilistic Machine Learning: Advanced Topics* (MIT Press, 2023) p. 499.

[5] J. You, J.-W. Chung, and M. Chowdhury, Zeus: Understanding and optimizing {GPU} energy consumption of {DNN} training, in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)* (2023) pp. 119–139.

[6] K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, in *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016) pp. 770–778.

[7] O. Ronneberger, P. Fischer, and T. Brox, U-net: Convolutional networks for biomedical image segmentation, in *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18* (Springer, 2015) pp. 234–241.

[8] B. Dai and D. Wipf, Diagnosing and enhancing VAE models, in *International Conference on Learning Representations* (2019).

[9] C. Chadebec, L. Vincent, and S. Allassonniere, Pythae: Unifying generative autoencoders in python-a benchmarking use case, Advances in Neural Information Processing Systems **35**, 21575 (2022).

[10] P. Ostheimer, M. Nagda, M. Kloft, and S. Fellenz, Sparse data generation using diffusion models, arXiv preprint arXiv:2502.02448 (2025).

[11] X. Liu, X. Zhang, J. Ma, J. Peng, *et al.*, Instaflow: One step is enough for high-quality diffusion-based text-to-image generation, in *The Twelfth International Conference on Learning Representations* (2023).

[12] D. Levin, Y. Peres, and E. Wilmer, *Markov Chains and Mixing Times* (American Mathematical Soc., 2009).

[13] D. P. Kingma and P. Dhariwal, Glow: Generative flow with invertible 1x1 convolutions, in *Advances in Neural Information Processing Systems*, Vol. 31, edited by S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Curran Associates, Inc., 2018).

[14] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan, Normalizing flows for probabilistic modeling and inference, Journal of Machine Learning Research **22**, 1 (2021).

[15] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, Generative adversarial nets, in *Advances in Neural Information Processing Systems*, Vol. 27, edited by Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger (Curran Associates, Inc., 2014).

[16] N. Freitas, G. Massarelli, J. Rothschild, D. Keane, E. Dawe, S. Hwang, A. Garlapati, and T. McCourt, *Taming non-equilibrium thermal fluctuations in subthreshold CMOS circuits*, Phys. Rev. Lett. (2025), Submitted.

[17] M. Sajeeb, N. A. Aadit, T. Wu, C. Smith, D. Chinmay, A. Raut, K. Y. Camsari, C. Delacour, and T. Srimani, Scalable connectivity for ising machines: Dense to sparse, arXiv preprint arXiv:2503.01177 (2025).

[18] A. Decelle, C. Furtlehner, and B. Seoane, Equilibrium and non-equilibrium regimes in the learning of restricted boltzmann machines*, Journal of Statistical Mechanics: Theory and Experiment **2022**, 114009 (2022).